



中国研究生创新实践系列大赛

中国光谷·“华为杯”第十九届中国研究生

数学建模竞赛

学    校    北京理工大学

---

参赛队号    22100070171

---

1.夏云卿

---

队员姓名    2.陈雅洁

---

3.姚运宝

---

---

# 中国研究生创新实践系列大赛

## 中国光谷·“华为杯”第十九届中国研究生

### 数学建模竞赛

#### 题 目      方形件排样优化与订单组批问题探析

---

#### 摘            要：

方形件排样优化与订单组批问题是计算复杂度很高的组合优化问题，在工业工程中有很广泛的应用背景。为实现个性化定制生产模式，企业会选择订单组批的方式，继而通过排样优化实现批量切割，加工完成后再按照不同客户需求进行分拣，从而提高原材料的利用率，进而降低生产成本，提高企业竞争力。

本文以二维方形件的排样优化及订单组批为对象，充分考虑各种约束条件，对两个问题分别建立混合整数规划模型，合理规划方形件在板材原片上的布局，整体协调订单分配与批量计划的集成优化，最终取得全局近似最优解。

针对问题一：优化目标是基于三阶段齐头切精确排样方式，在各产品项数量均不超过其需求量的约束下实现使用的板材原片数量最少。我们转“割”为“拼”，采用“均匀栈的生成→条带生成→原片条带放置”的步骤，设计了**均匀三阶段排样方式的生成算法（TUSP法）**得到满足条件的排样图。之后采用**顺序价值修正启发式算法（SVC法）**，逐个将满足条件的排样图加入到整个数据的排样方案中进行优化，最终得到基于整个数据的最优排样方案，得到 A1~A4 数据集的板材利用率依次为 **96.02%、94.08%、95.15%和 94.08%**。根据获得的各产品项的 x、y 方向坐标及度量，绘制各原片的排样方案效果图。

针对问题二：优化目标是通过订单组批的形式将订单聚类，并按照材质选择相应的排样方式，进而实现使用的板材原片数量最少。本问题的关键在于订单组批，后续排样过程可参考问题一。首先使用**杰拉德距离**定义了订单之间材质的相似性，以此作为**层次聚类**的依据，从而划分项目的组批。其次，对同一批次中的不同材质，依次对产品项使用**均匀三阶段排样方式的生成算法（TUSP法）**和**顺序价值修正启发式算法（SVC法）**获得最终的排样方案，得到 B1~B5 数据集的板材利用率依次为 **84.42%、82.84%、82.11%、83.44%和 83.51%**，并根据排样方案中产品项的坐标等信息绘制各原片的排样方案效果图。

最后，本文对所用算法的复杂度进行了评估，同时对模型的优缺点进行了分析。

**关键词：**混合整数规划模型；均匀三阶段排样方式的生成算法；顺序价值修正启发式算法；订单组批；层次聚类

---

## 目录

|                                       |    |
|---------------------------------------|----|
| 一、 问题重述.....                          | 3  |
| 1.1 问题背景.....                         | 3  |
| 1.2 问题提出.....                         | 3  |
| 二、 问题分析.....                          | 4  |
| 2.1 问题一分析.....                        | 4  |
| 2.2 问题二分析.....                        | 4  |
| 三、 模型假设.....                          | 5  |
| 四、 符号说明.....                          | 5  |
| 五、 问题一：模型建立与求解.....                   | 6  |
| 5.1 整数规划模型建立.....                     | 6  |
| 5.2 算法设计与建立.....                      | 7  |
| 5.2.1 均匀三阶段排样方式的生成算法（TUSP 法）.....     | 7  |
| 5.2.2 顺序价值修正启发式算法（SVC 法）.....         | 12 |
| 5.3 模型求解.....                         | 14 |
| 5.3.1 初始化设置.....                      | 14 |
| 5.3.2 求解过程.....                       | 15 |
| 5.4 求解结果与分析.....                      | 16 |
| 5.4.1 对于数据集 A1、A2、A3、A4 的排样方案效果图..... | 16 |
| 5.4.2 结果分析.....                       | 17 |
| 六、 问题二：模型建立与求解.....                   | 19 |
| 6.1 混合整数规划模型建立.....                   | 19 |
| 6.2 算法设计与建立.....                      | 19 |
| 6.2.1 订单凝聚层次聚类算法.....                 | 19 |
| 6.3 模型求解.....                         | 21 |
| 6.4 求解结果与分析.....                      | 21 |
| 6.4.1 对于数据集 B1、B2、B3、B4 的排样方案效果图..... | 21 |
| 6.4.2 结果分析.....                       | 25 |
| 七、 模型分析与评价.....                       | 25 |
| 7.1 算法复杂度分析.....                      | 25 |
| 7.2 模型优缺点分析.....                      | 26 |
| 7.2.1 模型优点分析.....                     | 26 |
| 7.2.2 模型缺点分析.....                     | 26 |
| 八、 参考文献.....                          | 26 |
| 九、 附录.....                            | 27 |
| 9.1 针对数据集 A1、A2、A3、A4 的完整排样方案效果图..... | 27 |
| 9.2 代码及注释.....                        | 44 |

## 一、问题重述

### 1.1 问题背景

新时期我国经济发展面临的重大课题，即是实现由制造大国向制造强国的转变，其中智能制造又是“中国制造 2025”的主攻方向。企业若想在智能制造转型中获取竞争优势，则需要在个性化定制方面提高竞争力。面对层出不穷的客户需求，如何在把控产品质量的同时预测订单规模，以便可以及时做出响应，就成为企业需要思考的问题。

方形件产品（或称板式类产品）即为依赖于个性化定制生产模式的一类产品，它以板材为主要原片，经过平面加工后，由几种板式配件装配而成。为了实现“多品种小批量”的个性化定制生产，面对数量庞大的订单，企业通常会通过订单组批的方式来实现批量切割，加工完成后再按照不同客户需求进行分拣，从而提高原料的利用率<sup>[1]~[3]</sup>。

在上述“订单组批+批量生产+订单分拣”模式中，订单组批与排样优化为两个重要环节。订单组批就是将相同材质、相近交货期、相似工艺的订单安排在同一生产批次，通过订单组批优化兼顾个性化需求与生产高效性（如组批批次太小，会造成材料利用率低与生产效率低；如组批批次太大，会影响生产效率从而影响订单交付）；排样优化本质为一个面向客户订单的材料切割问题，需要合理规划方形件在板材上的布局，以达到降低板材浪费、简化切割过程的目的。

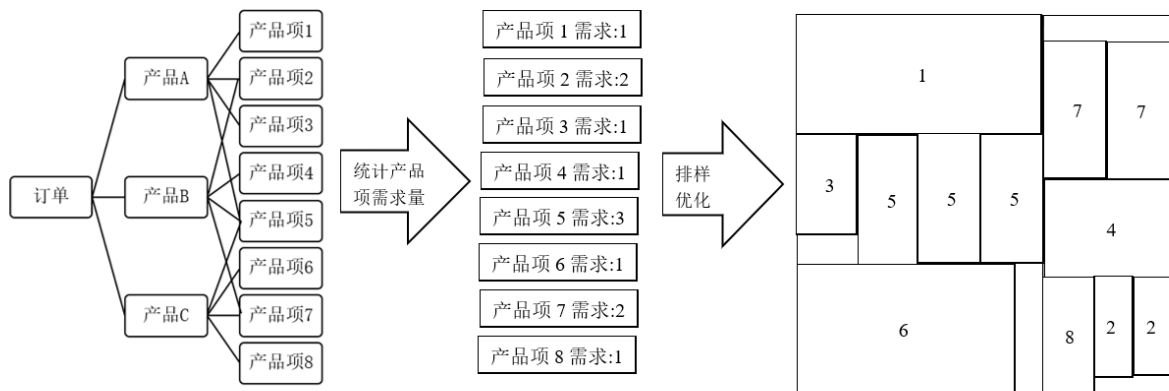


图 1 订单组批与排样优化过程图

对于排样优化问题，根据切割工艺方式的不同可分为齐头切与非齐头切（齐头切即直线切割且切割方向垂直于一边，并保证每次切割都可将原片分为两块），齐头切又可分为精确方式与非精确方式（区别在于是否可切割出准确尺寸的方形件）。三阶段齐头切主要有三种类型：三阶段非精确排样方式、三阶段匀质排样方式以及三阶段同质排样方式，后两种均为精确排样方式。基本经过三到四个阶段的切割即可满足客户需求。

### 1.2 问题提出

基于对订单组批与排样优化的理解，题目提出以下 2 个子问题，其中第 2 个子问题的约束都基于第 1 个子问题并增加了单个批次生产原片数和面积的限制：

**问题一：**排样优化问题。只考虑齐头切的切割方式且切割阶段数不超过 3 的精确排样，建立混合整数规划模型<sup>[4]</sup>，在满足订单需求及相关约束条件下，达到原片的最大化利用。

需要考虑的约束条件如下：

- 1) 在相同栈（stack）里的产品项（item）的宽度（或长度）应该相同；
- 2) 最终切割生成的产品项是完整的，非拼接而成。

**问题二：**订单组批问题。要求建立混合整数规划模型，对数据集 B 中全部的订单进行组批，然后对每个批次进行独立排样，在满足订单需求和相关约束条件下，使得板材原片的用量尽可能少。在满足子问题 1 约束的基础上进一步要求：

- 1) 每份订单当且仅当出现在一个批次中；
- 2) 每个批次中的相同材质的产品项 (item) 才能使用同一块板材原片进行排样；
- 3) 为保证加工环节快速流转，每个批次产品项 (item) 总数不能超过限定值；
- 4) 因工厂产能限制，每个批次产品项 (item) 的面积总和不能超过限定值。

## 二、问题分析

### 2.1 问题一分析

问题一需要建立混合整数规划模型得出三阶段齐头切精确排样方案，该方案由排样方式组成，排样方式为该排样优化问题的解。对于三阶段切割方式，要求同一阶段切割方向相同，相邻阶段切割方向互相垂直，我们并不限制第一阶段的切割方向，横向或纵向皆可，并称第一阶段切割生成的模块为 stripe(条带)，第二阶段切割生成的模块成为 stack(栈)，第三阶段切割生成的模块为 item(产品项)。实际应用中，产品项方向可以是固定的，也可以是旋转的，为了更贴近实际情况，我们考虑产品项方向可旋转。产品项的价值一般为其面积，且会赋予面积大的产品项一个较大的权重，以便优先考虑其放置。

排样方式必须满足：（1）产品项的个数不超过需求量；（2）排入的产品项之间互不重叠且不超过原片边界；（3）满足题目要求的三阶段齐头切精确排样方式。

本问题建立在三阶段匀质排样的基础上，即认为每个栈都由一组宽度相同的产品项组成，对比于三阶段同质排样，匀质排样的材料利用率可能会更高，但切割复杂度也会随之增加。

根据待切割的产品项的种类和数量，确定产品项在原片上的排列组合方式，在满足产品项数量与工艺约束的条件下，减少原材料的损耗。此排样优化问题可描述为：在尺寸为  $L \times W$  的原片上寻找最佳排样方案，排入  $n$  种产品项，其中第  $i$  项产品项的长、宽及需求量为  $(l_i \times w_i \times d_i)$ ， $\sum_{j=1}^{i'} \alpha_{ji'} = 1, \forall i = 1, \dots, 2n, i \in I = \{1, \dots, n\}$ ，需要使消耗的原片成本最低，且排样方式需符合切割工艺要求。

本问题的优化目标是在各产品项数量均不超过其需求量的约束下实现原片中包含的产品项总价值最大，可以将目标设定成以板材用量最少，另一种角度是使残料最少。尽管追求板材用量最少与追求残料最少的实现方式是一致的，但后者方法的灵敏度较差，最终可能会导致无解；且对于残料，如果其尺寸足够大，就可以成为余料二次利用，所以实际问题中考虑残料最少并不合理。

### 2.2 问题二分析

问题二需要建立混合整数规划模型，对数据集 B1~B5 中全部的订单进行组批，并对每个批次进行独立排样，在满足订单需求和相关约束条件下，使得板材原片的用量尽可能少。该方案由组批方式组成，组批方式为该子问题的解。

组批方式必须满足：（1）每份订单当且仅当出现在一个批次中；（2）每个批次可生产的产品项总数不超过上限；（3）每个批次可生产的产品项面积总和不超过上限。

订单组批问题是一个 NP 完全问题，求其精确解是比较困难的。模型的优化目标是最小化板材原片使用数量，可以考虑在组批过程中尽量将使用同种材质的订单放在同一批次，这样同一批次中相同材料的产品项种类和数量也越多，在之后的排样过程中，也更容易得

到更高的板材使用率，从而达到目标。所以该子问题的关键就转为寻求一个合适的差异度函数来度量订单间的相似程度，在此基础上对订单进行聚类，最终生成分批后的订单。后续排样过程与第一问类似，可按照第一问的思路进行排样优化。

### 三、模型假设

- 假设 1: 本题假定排样优化方式为三阶段齐头切匀质排样，且产品项作为切割的最小单位需保持完整；
- 假设 2: 本题假定第一阶段切割方向为横向的排样方式为 X 向排样，第一阶段切割方式为纵向的排样方式为 Y 向排样；
- 假设 3: 本题假定板材原片仅有一种规格，即原片长度为 2440 (mm)，原片宽度为 1220 (mm)，且原片数量充足；
- 假设 4: 本题假定排样方案不受锯缝宽度影响；
- 假设 5: 本题假定在切割过程中不存在设备故障、生产事故；
- 假设 6: 本题假定所有订单的交货期均相同，即订单组批问题不考虑交货期因素；
- 假设 7: 订单包含的产品项的种类和数量满足某种随机分布；
- 假设 8: 一个订单仅可组批一次，且不能跨批次组批；
- 假设 9: 批次与批次之间不会互相交叉混合。

### 四、符号说明

本文所用符号的说明如下表所示。

**指标:**

| 符号  | 说明                                                                                                                                    |
|-----|---------------------------------------------------------------------------------------------------------------------------------------|
| $i$ | 产品项编号的指标，对 $n$ 个产品项按宽度进行排序 ( $w_1 \geq w_2 \geq \dots \geq w_n$ )，即产品项 1，产品项 2， $\dots$ ，产品项 $n$ 将宽度不增，其中 $i \in I = \{1, \dots, n\}$ |
| $j$ | 栈编号的指标，将栈中所含产品项中宽度最大的产品项编号作为栈的编号，其中 $j \in J = \{1, \dots, n\}$                                                                       |
| $k$ | 条带编号的指标，将条带中所含栈的最宽栈的编号作为条带编号，其中 $k \in K$                                                                                             |
| $l$ | 原片编号的指标，将原片中所含条带的最小索引作为原片编号，其中 $l \in L$                                                                                              |

**输入参数:**

| 符号  | 说明            |
|-----|---------------|
| $L$ | 原片长度 (单位: mm) |
| $W$ | 原片宽度 (单位: mm) |

|                |                                                                               |
|----------------|-------------------------------------------------------------------------------|
| $l_i$          | 产品项 $i$ 的长度（单位：mm）                                                            |
| $w_i$          | 产品项 $i$ 的宽度（单位：mm）                                                            |
| <b>决策变量：</b>   |                                                                               |
| 符号             | 说明                                                                            |
| $\alpha_{ji}$  | 产品项 $i$ 在栈 $j$ 中时取 1，否则为 0， $j \in \{1, \dots, n\}$ ， $i \in \{j, \dots, n\}$ |
| $\beta_{kj}$   | 栈 $j$ 在条带 $k$ 中时取 1，否则为 0， $k \in \{1, \dots, n\}$ ， $j \in \{1, \dots, n\}$  |
| $\gamma_{lk}$  | 条带 $k$ 在原片 $l$ 中时取 1，否则为 0， $l \in \{1, \dots, n\}$ ， $k \in \{l, \dots, n\}$ |
| $\delta_{lij}$ | 产品项 $i$ 不是原片 $l$ 中最宽条带中最宽栈中的最宽产品项时取 1，否则为 0                                   |

## 五、 问题一：模型建立与求解

### 5.1 整数规划模型建立

根据题目要求建立整数规划模型<sup>[8]-[10]</sup>如下：

对于问题一，可以理解成有约束条件的三阶段二维装箱问题。装箱问题就是我们有  $n$  个小矩形，以及无限个大矩形箱子，目标是将全部小矩形无重叠的放入箱子中，小矩形放入箱子的方向可以旋转，水平垂直均可。三阶段切割可以理解：1、沿第一个方向齐头切割后，将矩形切割成几个条带，2、沿第二个方向齐头切割后，将条带切割成几个栈，3、沿第三个方向齐头切割后，将栈切割为产品项。显然，由同一个栈切割得到的几个产品项具有相同的宽度。为统一规则，便于切割，我们将矩形尽量放置于大矩形箱子的左下位置，使得如果必须存在剩余空间的话，剩余空间位于右上位置。由于几个产品项组成一个栈后被放置于大矩形箱子内，故产品项排列的前后顺序对栈在大矩形箱子内的摆放位置并无影响，我们可以将同一个栈内的几个产品项按照产品项的索引从小到大排列，便于后续处理。产品项组成栈后，我们可以获得一定数量的栈，栈组成条带与产品项组成栈类似，同一条带内栈的排序也可以前后调整。对栈进行编号索引，类似的，我们也可以在条带内将不同的栈按照索引顺序从小到大排列。这种排序方法有利于后续我们得到每个产品项在大矩形箱子的位置。最后再将条带放置于大矩形箱子内，找到最好的放置方式使得只需要尽可能少的箱子数即可放置下全部条带。

针对产品项可旋转，我们考虑将  $n$  个产品项复制一份，并将其长宽互调，得到了  $2n$  个产品项，并进一步按照产品项的宽度进行排序，这时使用两个映射  $f$  和  $g$ ， $f$  表示目前排序下产品项对应为复制前的该产品项索引编号， $g$  表示产品项和其复制品的索引编号差。

$$\min \sum_{i=1}^n \gamma_{rr} \quad (1)$$

s.t.

$$\sum_{j=1}^{i'} \alpha_{ji'} = 1, \forall i = 1, \dots, 2n, \quad (2)$$

$$\sum_{i'=j+1}^n \alpha_{ji'} \leq (2n-j)\alpha_{jj}, \forall j = 1, \dots, 2n-1, \quad (3)$$

$$\alpha_{ji'} = 0, l_{f(i')} \neq l_{f(j')}, w_{f(i')} + w_{f(j')} = W, \forall i' > j, \forall j = 1, \dots, 2n-1, \quad (4)$$

$$\sum_{k=1}^j \beta_{kj} = \alpha_{jj}, \forall j = 1, \dots, 2n, \quad (5)$$

$$\sum_{i'=j}^{2n} w_{f(i')} < \sum_{i'=k}^{2n} w_{f(i)} \alpha_{ki'} + (W+1)(1-\beta_{kj}), \forall k = 2, \dots, n, \forall j = 1, \dots, k-1, \quad (6)$$

$$\sum_{i=j}^{2n} w_{f(i)} \alpha_{ji'} \leq \sum_{i'=k}^{2n} w_{f(i)} \alpha_{ki'} + W(1-\beta_{kj}), \forall k = 1, \dots, 2n-1, \forall j = k+1, \dots, 2n, \quad (7)$$

$$\sum_{j=1}^{2n} l_j \beta_{kj} \leq l \beta_{kk}, \forall k = 1, \dots, 2n, \quad (8)$$

$$\sum_{l=1}^k \gamma_{lk} = \beta_{kk}, \forall k = 1, \dots, 2n, \quad (9)$$

$$\sum_{i'=1}^{2n} w_{f(i')} \gamma_{li'} + \sum_{i'=l+1}^{2n} w_{f(i)} \sum_{j=1}^{i'-1} \delta_{li'j} \leq W \gamma_{ll}, \forall l = 1, \dots, 2n-1, \quad (10)$$

$$\alpha_{ji'} + \gamma_{lj} - 1 \leq \delta_{li'j} \leq \frac{\alpha_{ji'} + \gamma_{lj}}{2}, \forall l = 1, \dots, 2n-1, i = l+1, \dots, 2n, j = l, \dots, i+1, \quad (11)$$

$$\sum_{k=l+1}^{2n} \gamma_{lk} \leq (n-l) \gamma_{ll}, \forall l = 1, \dots, 2n-1, \quad (12)$$

$$\sum_{i'=1}^{2n} [\alpha_{ji'} + \alpha_{j(i'+g(i'))}] = 1, \forall i' = 1, \dots, 2n. \quad (13)$$

## 5.2 算法设计与建立

### 5.2.1 均匀三阶段排样方式的生成算法（TUSP 法）

均匀三阶段排样方式即为“均匀栈的生成→条带生成→原片条带放置”三个步骤，下面我们将分别介绍这三阶段的模型与算法，并在最后给出均匀三阶段排样方式的生成算法（Three Uniform Stripe Pattern, 后简称为 TUSP）

#### 1. 有约束的均匀栈的生成模型及算法

##### 1) 均匀栈的价值确定及生成模型



有约束限制的均匀栈的生成算法，需要考虑到每种栈的需求约束。我们考虑主产品项这个概念，对于每个栈都会有一个产品项被选为主产品项，主产品项决定了水平栈的宽度或者竖直栈的长度。在本文中，我们选取水平栈的位置最左边的产品项作为主产品项，竖直栈的位置最下边的产品项作为主产品项。对于生成主产品项为 $i$ 的水平均匀栈，记之为 $S_i$  'h',  $w_i, L$ ，其中栈长度不大于 $L$ ，宽度等于 $w_i$ ，'h'表示此栈水平放置。对于生成主产品项为 $i$ 的竖直均匀栈，记之为 $S_i$  'v',  $l_i, W$ ，其中栈宽度不大于 $W$ ，长度等于 $l_i$ ，'v'表示此栈竖直放置。下面给出生成主产品项为 $j$ 的水平均匀栈的数学模型，如下

$$V_j = \sum_{i=1}^m c_i \times z_{ji} \quad (14)$$

s.t.

$$\sum_{j=1}^m z_{ji} \leq d_j, \quad (15)$$

$$w_j = w_i \mid w_j \in S_i, \quad (16)$$

$$\sum_{j=1}^m l_i \times z_{ji} \leq L. \quad (17)$$

上式中， $V_j$ 为此水平栈的价值， $c_j$ 为产品项 $i$ 的价值， $z_{ji}$ 为矩形产品项 $i$ 在水平栈 $S_j$ 中出现的次数， $d_i$ 为产品项 $i$ 的需求约束量，水平栈 $S_j$ 的宽度等于 $w_j$ ，长度小于 $L$ 。

## 2) 均匀栈的生成算法

根据上一节中建立的均匀栈的生产模型，再结合每个产品项的需求约束，我们可以将此问题转化成0-1背包问题，并且使用背包算法进行求解。但由于产品项的种类数量较多，且随着产品项的排入导致的产品项的数量上限的变化，这会使得算法的时间复杂度骤升。因此，在本文中我们考虑使用贪心算法来对均匀栈进行生成。

### Step1. 算法初始化设置

**产品项可旋转：**为此我们选择将所有产品项复制一份，并将其长宽互调，从而现在的产品项共有 $2m$ 个，由于一个产品项只能被选择一次，故在后续算法中需添加产品项和它的复制品只能选择一个进行排样。

**终止条件的设置：**在使用贪心算法生成栈时，需要一些终止条件来判断能否继续向栈排入产品项，本算法终止条件包含如下两个：

➤ 栈的剩余宽度（竖直栈）或者剩余长度（水平栈）不能在放入任何产品项；

注：对于此终止条件，若对每一个未排入的产品项与剩余宽度或剩余长度进行一对一的顺序比较，则会大幅增加算法的时间复杂度。为了减少算法的计算复杂度，我们考虑建立两个一维数组向量 $lmin$ 和 $wmin$ ，记录产品项的长度和宽度的最小值，其计算表达式如下：

$$l \min(i) = \min\{l_k, |i \leq k \leq 2m, k \in N^*\}, \quad (18)$$

$$w \min(i) = \min\{w_k, |i \leq k \leq 2m, k \in N^*\}. \quad (19)$$

其中  $l_k$  和  $w_k$  表示产品项  $k$  的长度和宽度,  $l \min(i)$  和  $w \min(i)$  表示产品项  $i$  到  $m$  的产品项长度最小值和宽度最小值。这时在判断能否继续向栈排入产品项时, 仅需将  $l \min(i)$  和  $w \min(i)$  分别与剩余长度和剩余宽度比较即可, 这大大减少了算法了计算复杂度。

➤ 产品项已经耗尽;

注: 对于此终止条件, 我们仍考虑建立一个一维数组  $demand$  对每个产品项的剩余量进行记录, 这是再判断某产品项是否有剩余可排入栈时, 仅需与  $demand(i)$  进行对比即可。

**初始化  $demand$  数组:**  $demand(i)$  表示产品项  $i$  在栈中排入的限制数量, 令其初值等于产品项的初始需求, 其赋值计算表达式如下:

$$demand(i) = d(i), \quad i = 1, \dots, m. \quad (20)$$

其中  $d(i)$  表示产品项的初始需求。

### **Step2.** 算法建立过程

在将产品项排入栈的过程中, 需要对生成的栈的信息进行存储, 我们考虑建立一个一维数组向量对栈  $j$  的信息存储, 向量构建如下:

$$m_j = \{w, (1, num_{j1}), \dots, (i, num_{ji}), \dots, (2m, num_{j2m})\}. \quad (21)$$

其中  $w$  表示此栈的放置方式,  $w = 'h'$  表示水平放置,  $w = 'v'$  表示竖直放置, 二元数组  $(i, num_{ji})$  中,  $i$  表示产品项的编号,  $num_{ji}$  表示此产品项在栈中排入的次数。

在得到栈的主要信息之后, 我们需要进一步计算栈的价值。并且由于生成的是均匀栈, 故此要求栈的所有出现的产品项的宽度或者长度均是相等的。下以水平均匀栈为例: 设以产品项  $j$  作为主产品项的水平均匀栈  $j$ , 其长度不大于  $L$ , 且宽度等于  $w_j$ , 记此栈的价值为

$HV(j, L, w_j, m_j)$ , 其求解的伪代码如下:

---

---

### 算法 1：均匀栈生成算法

---

**输入：** 栈价值  $V$ ；条带剩余长度  $RL$ ；栈中包含产品项  $i$  的数量  $num$

**输出：** 栈价值  $HV(j, L, w_j, m_j)$ ；栈的数组信息  $m_j$

---

```
1   $RL = L - l_j$ ;  
2  for  $i=1$  to  $2m$   
3      if  $RL < l_{\min(i)}$  or  $l_i \neq l_j$   
4           $HV(j, L, w_j, m_j) = V$ ; break  
5      else  
6          if  $demand(i) = 0$  return  
7          else  
8              if  $RL < l_i$  return  
9              else  
10                  $num = \min(demand(i), \lfloor RL/l_i \rfloor)$ ;  $RL = L - num \times l_i$   
11                 Add  $(i, num)$  to  $m_j$ ;  $V = V + num \times c_i$   
12             end  
13         end  
14     end  
15 end  
16 return  $HV(j, L, w_j, m_j)$ ;  $m_j$ 
```

---

## 2. 条带生成算法

我们类比上一节提出的产品项排成栈的算法，进一步将栈排条带中，从而可得到由栈生成条带的算法，由于和栈生成算法类似，下面仅简略展示条带生成算法的步骤，如下：

**Step1.** 算法初始化设置

**确定栈的各类属性：** 首先通过使用栈生成算法生成了  $n$  个栈，进而通过算法输出结果计算水平栈的长度和竖直栈的宽度，并将其储存。

**确定栈的使用需求：** 在这部分我们认为算法生成的  $n$  个栈均是不同的，即当两个栈的长宽以及内部包含的产品项种类和数目均相同时，而这时栈内产品项的排列顺序也可以是不同的，故此我们认为生成的  $n$  个栈均是不同的。进而设置每个栈的需求量均为 1。

**Step2.** 算法建立过程：

在这部分我们类比产品项生成栈的算法，进行算法设计，将  $n$  个栈放置进条带中，具体流程均与上节所述类似，在此不进行重复展示。

### 3. 原片放置条带算法

原片的价值最大即原片上所排样的各条带的价值之和最大，为此下面我们进一步将条带排进原片中。在这部分我们只给出水平条带排样成原片的算法，竖直条带的完全类似。使用迭代，首先设  $Y \in \{y | 1 \leq y \leq W, y \in N^*\}$  为原片的迭代宽度，用  $Z(l, Y)$  表示当前原片的尺寸为  $L \otimes Y$  时，在原片上放置前  $l$  个条带得到的最大价值，其中  $l=1, \dots, q$ ， $q$  为上一节中生成条带的数目。

**Step1.** 算法初始化设置

**条带的宽度度量设置：**我们选取的原片的迭代宽度均为整数，若条带的宽度不为整数，选用合适的数据处理方式，将所有条带的宽度均转换为整数形式。

**初始化迭代参数设置：**令  $Z(0, Y)=0$ ，其中  $Y=0, \dots, W$ ； $Z(k, 0)=0$ ，其中  $k=0, \dots, n$ 。

**Step2.** 迭代递推算算法过程

考虑计算当前尺寸为  $L \otimes Y$  的原片上放置前  $l$  种条带得到的原片最大值  $Z(l, Y)$ ，首先将其初始化为  $Z(l-1, Y)$ ，求出前放置前  $l-1$  个条带得到的原片最大值。这时再考虑能否将第  $l$  个条带放置进去，若此条带的宽度  $tw_l$  小于等于原片迭代宽度的  $Y$ ，则可以放置进去，进一步计算剩余原片的价值，即在尺寸为  $L \otimes (Y - tw_l)$  的原片上放置前  $l-1$  种条带得到的原片最大值，这时再加上方放置栈的价值  $HV(l, L, w_l, m_l)$ ，与  $Z(l-1, Y)$  进行对比，取最大值。反之，若此条带的宽度  $tw_l$  大于原片的  $Y$ ，即没法将条带  $l$  放置进至尺寸为  $L \otimes Y$  的原片，从而这时  $Z(l, Y)$  为  $Z(l-1, Y)$ 。由此方法，我们可以得到原片的一种近似排样方式。将其递推算表达式，如下：

$$Z(l, Y) = \max Z(l-1, Y), Z(l-1, Y - tw_l) + HTV(l, L, w_l, m_l) I_{\{Y \geq tw_l\}} \dots \quad (22)$$

其中  $tw_l$  表示条带  $l$  的宽度， $HTV(l, L, w_l, m_l)$  表示条带  $l$  的价值， $I_{\{Y \geq tw_l\}}$  为示性函数，

当  $Y \geq tw_l$  时，取 1，反之则取 0。

**Step3.** 原片的最优排样图

根据上述设置的初值，使用递推式进行迭代计算  $Z(n, W)$ ，此计算结果在尺寸为  $L \otimes W$  的原片上放置所有的  $q$  个条带得到的最大价值，即得到了一个原片的最优排样方式或排样图。

### 4. 均匀三阶段排样方式的生成算法（TUSP）

首先，通过 5.2.1 第 1 部分提出的均匀栈的生成算法，根据  $m$  个产品项的各种属性和信息，我们可以将其排样成  $n$  个栈，并储存生成栈的各种属性和信息。其次，通过 5.2.1 第 2 部分提出的条带生成算法，我们进一步可以将  $n$  个栈生成  $q$  个条带，并储存生成条带的各种属性和信息。最后，通过 5.2.1 第 3 部分提出的原片放置条带算法，我们就可以得到一张原片上产品项的最好排样方式（仅针对本文算法求得的最好结果）。

我们将上述三个算法进行结合，即得到了一种均匀三阶段排样方式的生成算法（在这里我们仅展示 X 向切割的排样方式的生成算法，Y 向切割类似，只要将原片的长宽互换即可），X 向切割的排样方式的生成算法具体步骤如下：

**Step1.** 根据均匀栈生成算法，生成  $n$  个栈，并确定栈的放置方式、个数、价值及其中排列的产品项的种类和数目。

**Step2.** 根据条带生成算法，生成  $q$  个条带，并确定条带的放置方式、个数、价值及其中排列的栈的种类和数目。

**Step3.** 根据原片放置条带算法，我们计算当前在尺寸为  $L \otimes W$  的原片上放置  $l$  个条带得到的最大价值  $Z(l, l)$ ，其中  $l=1, \dots, q$  和  $Y=1, \dots, W$ ，进而可计算得到在尺寸为  $L \otimes W$  的原片上放置  $q$  个条带的最大值。

**Step4.** 根据前三种算法计算得到的产品项在栈中排样、栈在条带中的排样、条带在原片中的排样，选择其中价值最大的排样方式，根据 Bottom-Left 原则（尽可能在左边和下边放置图形），我们可以绘制出此种排样方式的排样图。

TUSP 算法框图表示如下：

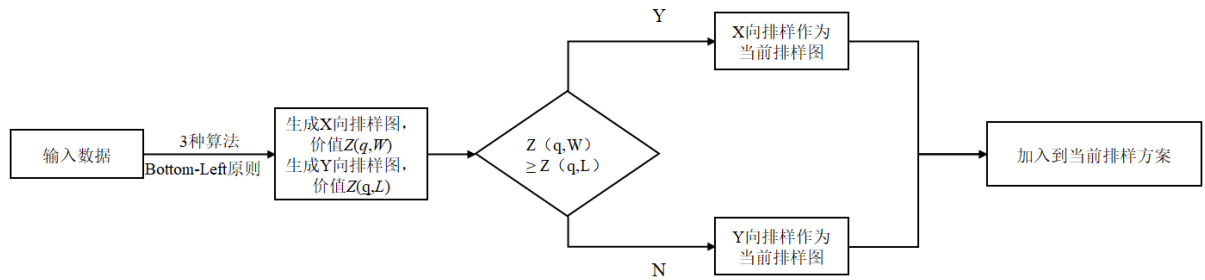


图 2 TUSP 算法框图

### 5.2.2 顺序价值修正启发式算法（SVC 法）

顺序启发式算法（SHP）Haessler 于 1975 年针对一维下料问题提出的算法<sup>[5]</sup>，后推广成为求解二维下料问题的一类方法，它的主要思想是利用当前产品项生成排样方式，并将其加入到排样方案中，再选中相应的产品项数量，然后重复此过程直到所有的产品项需求被满足。SHP 算法存在局部最优问题，且算法本身也具有贪婪性。在排样过程中，SHP 优先选取能获得高价值排样方式的产品项进行处理，那么随着排样方案中各排样方式的生成，剩余的产品项将越来越难处理，即后期生成的排样方式利用率低，相应的切割损失也极大。为了解决 SHP 算法存在的最优问题，进而得到接近于全局最优的排样方案，有学者提出了在顺序生成排样方式的过程中动态地对产品项的价值进行修正的方法，即顺序价值修正启发式算法<sup>[6]</sup>（SVC），它的主要思想是顺序生成当前排样方式后，修正当前排样图中出现的出现的产品项的价值，接着将新价值的产品项重新投入到下一轮排样中，经过多次的迭代，使产品项的价值趋于合理。SVC 框架有“多代”、“顺序”和“自适应修正”三个特点，如下：

- 多代：就是可以多次迭代，每代生成一个下料方案，在多个下料方案中选择使用板材数量最少的下料方案作为算法的解。
- 顺序：每生成一个排样方式，都可以满足部分毛坯的需求，将该排样方式加入到当前排样方案中，重复此过程直至所有的毛坯需求均得到满足。
- 自适应修正：在排样方式生成过程中，可不断的修正各个控制参数的值，以期得到最好排样方案。

SVC 算法框图表示如下：

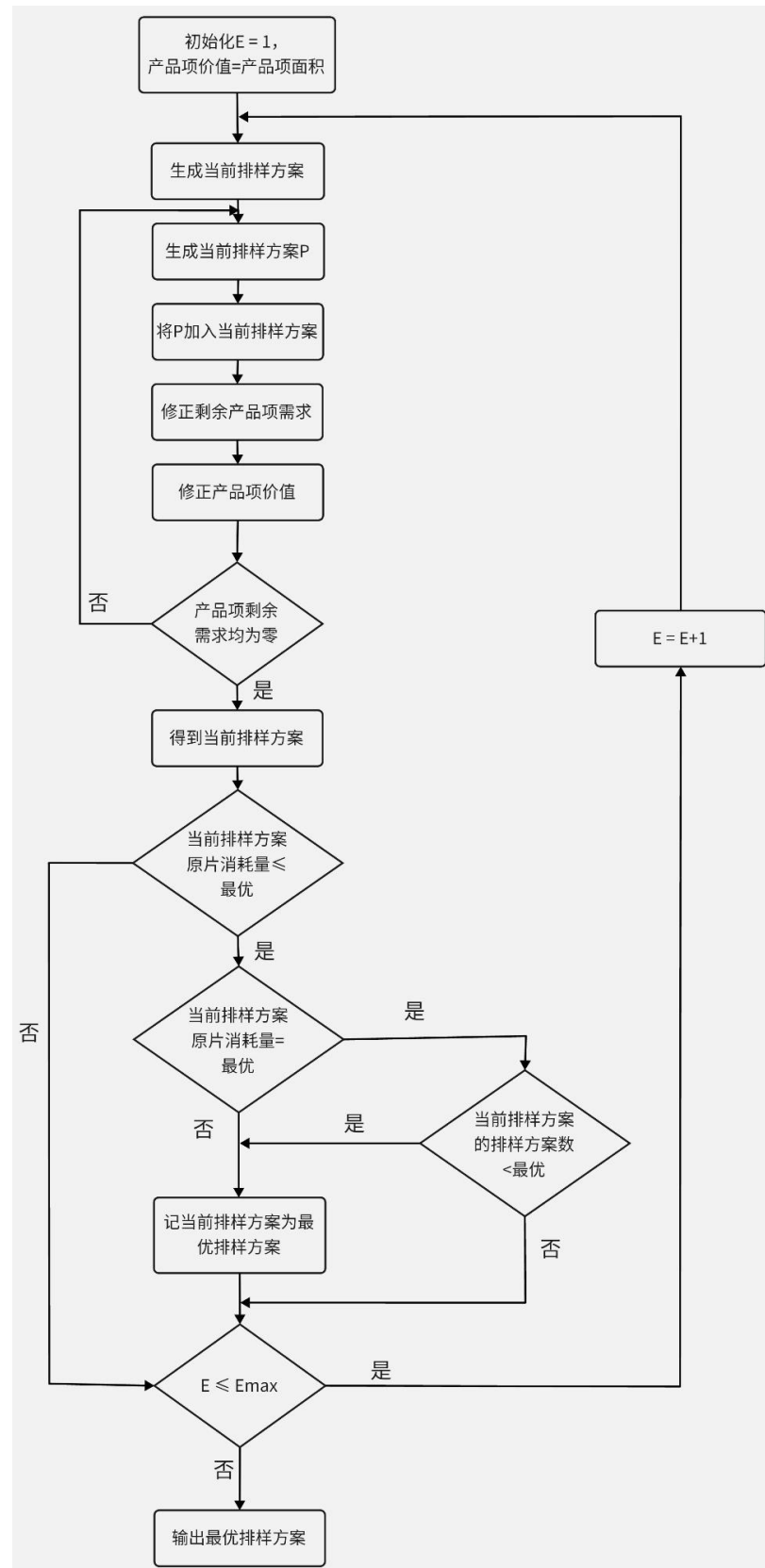


图 3 SVC 算法框图

在排样过程中，每生成一个排样方式就更新一次产品项的使用量并修正该排样方式中出现过的产品项的价值，重复此过程，直至所有的产品项需要均得到满足。在生成下一种排样方案时，各产品项的价值为上一种排样方案修正后的产品项价值。用  $E_{\max}$  表示生成的排样方案数， $E$  表示当前排样方案的序号。迭代  $E_{\max}$  次，得到  $E_{\max}$  中排样方案。比较各排样方案消耗的原片数，从中选择原片消耗量最少的排样方式作为最好排样方案。

本文中我们使用二维价值修正公式来进行求解，使用的公式如下：

$$\rho_i = \lambda_1 \rho_i + \frac{\lambda_2 (l_i w_i)^\tau}{\mu} \quad (23)$$

其中符号定义如下：

$\lambda_1$  为控制参数，计算表达式： $\lambda_1 = 1 - \eta r_i / (r_i + d_i)$ ；

$\lambda_2$  为控制参数，计算表达式： $\lambda_2 = 1 - \lambda_1$ ；

$\eta$  为参数，且  $\eta \in [0.6, 0.9]$ ，默认值为 0.75；

$r_i$  为排样图  $P$  中所含产品项  $i$  的个数， $i \in I$ ；

$d_i$  为产品项  $i$  的需求量， $i \in I$ ；

$\tau$  为略大于 1 的价值控制参数，默认值为 1.02；

$\mu$  为排样图  $P$  中所含产品项的总面积和原片面积之比，即板材利用率。

价值修正函数由修正前的产品项价值  $\rho_i$  和  $l_i w_i^\tau / \mu$  两部分组成，参数  $\lambda_1$ 、 $\lambda_2$  分别控制两部分的比重。 $l_i w_i^\tau / \mu$  表明：(1) 面积较大的产品项通常不利于组合，可通过适当的提高产品项价值来提升其优先级，使该产品项在后续的排样方式生成过程中能被优先考虑；(2) 若当前排样方式的利用率  $\mu$  较小，证明该排样方式中的产品项结合较差，应适当提高产品项价值，以提升其优先级。价值修正的目的是实现排样方案的多样化，选择不同的  $p$  值，对比各实验结果并从中选优。

### 5.3 模型求解

下面我们将基于 TUSP 和 SVC 算法求解该子问题。

#### 5.3.1 初始化设置

令产品项  $i$  的价值为  $\rho_i$ ，其中  $i \in I$ ； $U$  为当前排样方案使用的原片个数； $U_{op}$  为最好排样方案中使用的原片个数；产品项  $i$  的可用量为  $c_i$ ，其中  $i \in I$ ； $P$  为当前排样图，其中包

含产品项  $i$  的个数为  $r_i$ ，其中  $i \in I$ ； $J$  为当前排样方案的排样方式数， $J_{op}$  为最好排样方案的排样方式数。

### 5.3.2 求解过程

下面基于 TUSP 排样图生成算法和 SVC 顺序价值修正启发式算法来求解在产品项可旋转的条件下的三阶段匀质齐头切情况下二维矩形排样问题，模型的求解流程如下：

---

#### TUSP 和 SVC 求解 3CS 框图

---

**输入：** 初始化产品项单位价值，令  $\rho_i = l_i w_i$ ，其中  $i \in I$ ，令  $U_{op} = +\infty$

**输出：** 最优的排样方案

---

```

1      For  $E=1$  to  $E_{\max}$ 
2          初始化产品项可用量，令  $c_i = d_i$ ，其中  $i \in I$ 
3          令  $U=0$ ， $J=0$ 
4          While  $\sum_{i=1}^m c_i > 0$ 
5              使用 TUSP 算法生成当前排样图  $P$ 
6              确定  $P$  的使用次数，计算表达式为  $f = \min \lfloor c_i/r_i \rfloor | r_i \wedge i \in I$ 
7              将  $P$  加入当前排样方案
8              更新当前排样方案的排样方式数  $J$ ： $J=J+1$ 
9              更新原片使用数量  $U$ ： $U=U+1$ 
10             更新产品项可用量  $c_i$ ： $c_i = c_i - f \times r_i$ 
11             使用价值修正公式来修正产品项的价值
12         end
13         If  $U < U_{op}$ 
14             记当前排样方案为最优方案： $U_{op} = U$ ， $J_{op} = J$ 
15         end
16         If  $U_{op} = U$  and  $J_{op} > J$ 
17             记当前排样方案最优： $J_{op} = J$ 
18         end
19     end

```

---



5.4 求解结果与分析

5.4.1 对于数据集 A1、A2、A3、A4 的排样方案效果图

基于所建立的模型，使用 Python 绘出排样方案效果图，部分结果如下（由于排样方案效果图数量较多，故仅展示部分效果图，全部效果图见附录 9.1）：

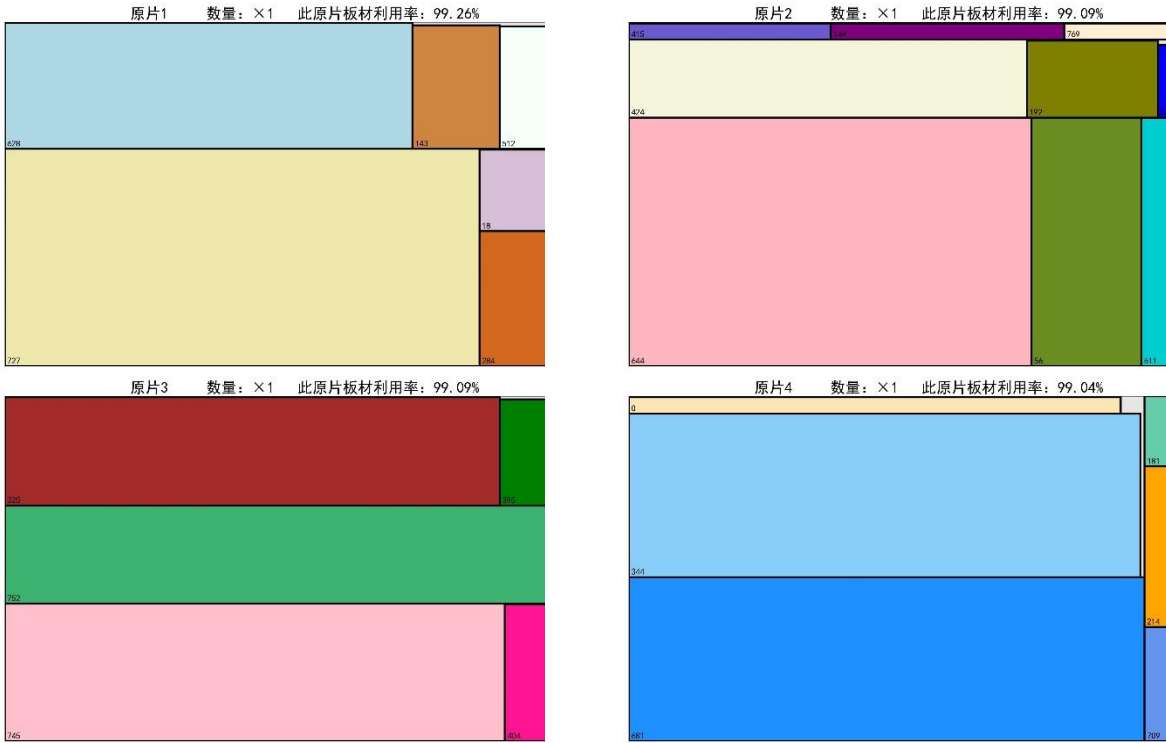


图 4 数据集 A1 的部分排样方案效果图

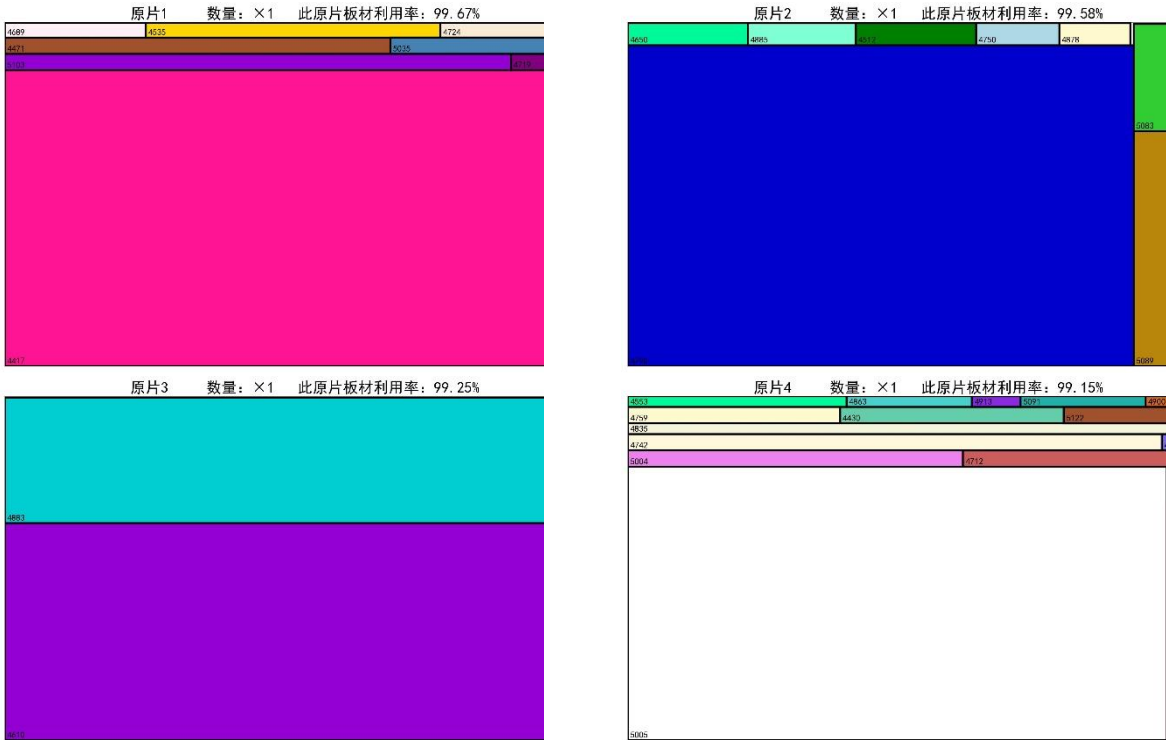


图 5 数据集 A2 的部分排样方案效果图

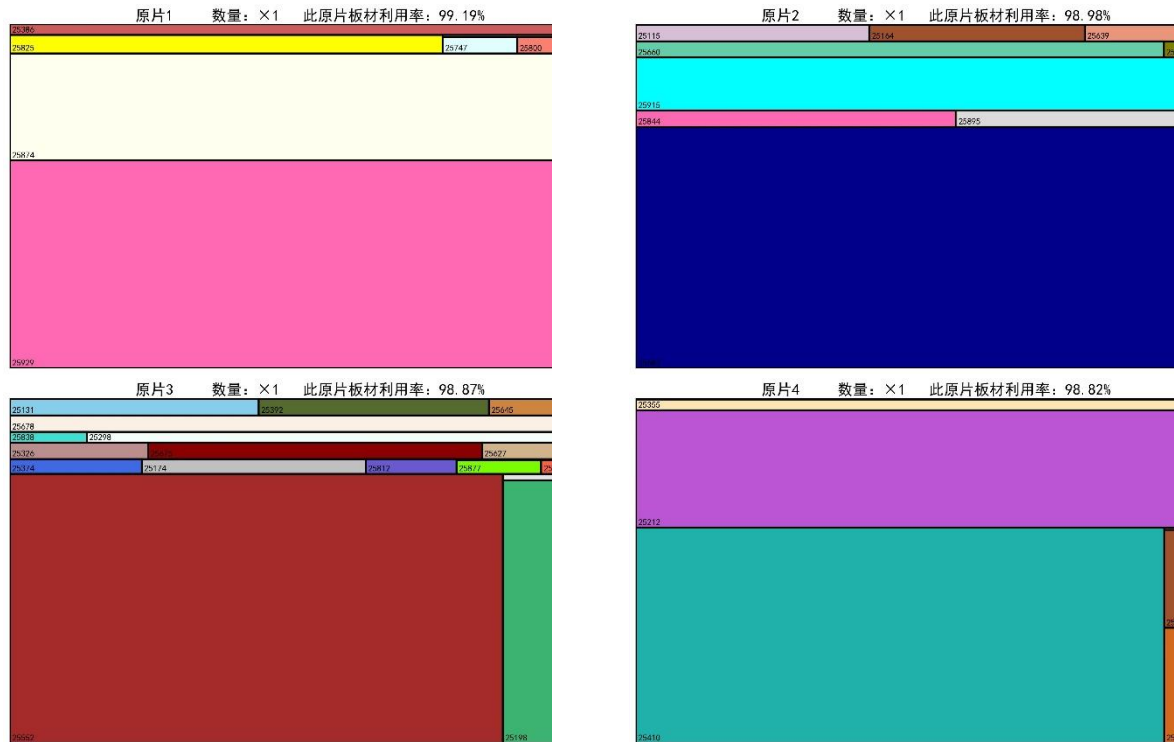


图 6 数据集 A3 的部分排样方案效果图

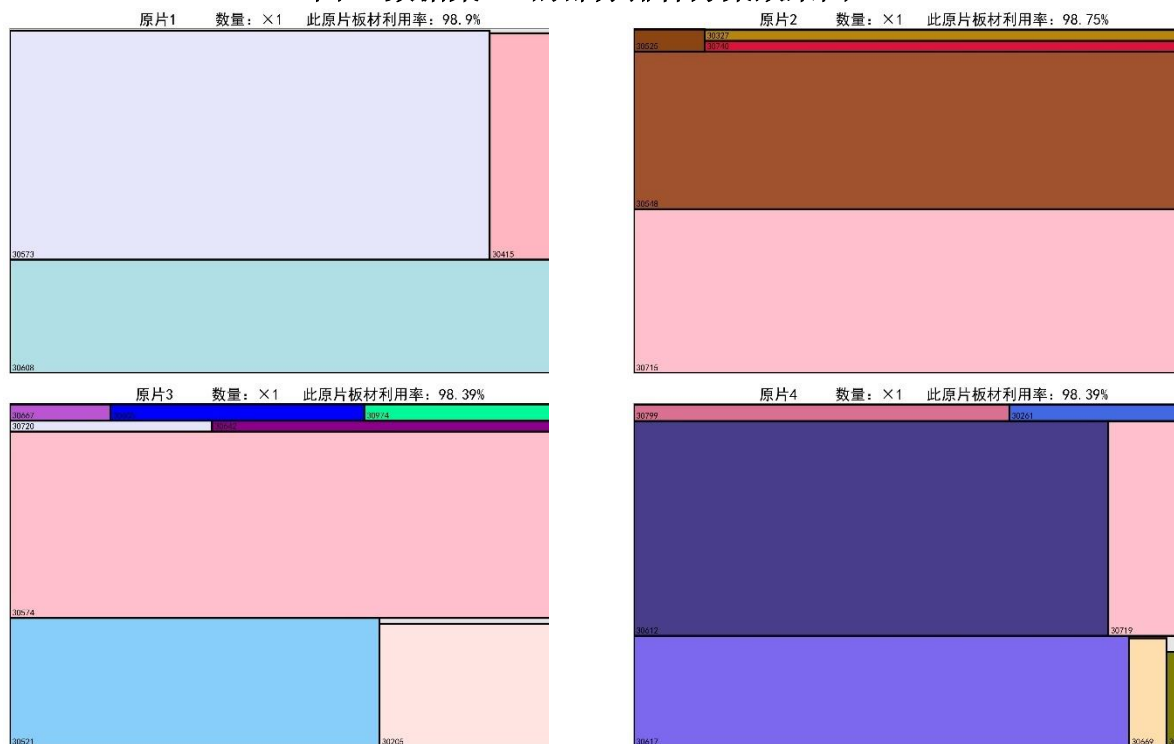


图 7 数据集 A4 的部分排样方案效果图

#### 5.4.2 结果分析

##### 1) 基于实际有效利用率与理论极限利用率的对比分析

在本问题中，可使用实际有效利用率来表示排样方案的效果，其计算公式可由数据集中所有产品项面积和与所用所有原片面积和的商值所定义，实际有效利用率越高说明排样方案越好。由于原片均为完整未切割原片，我们可计算理论极限利用率来进行比较，如对于数据集 A1，其中所有产品项面积和为 248.6856 平方米，需要 83.54 块原片，即至少需

要 84 块，则极限利用率为 99.45%，因此排样后实际有效利用率越接近 99.45%越好(理论不可能超过)。

由输出结果可知，基于建立的模型，数据集 A1 使用 87 块原片进行排样，数据集 A2 使用 87 块原片进行排样，数据集 A3 使用 88 块原片进行排样，数据集 A4 使用 86 块原片进行排样。利用率对比如下表所示。

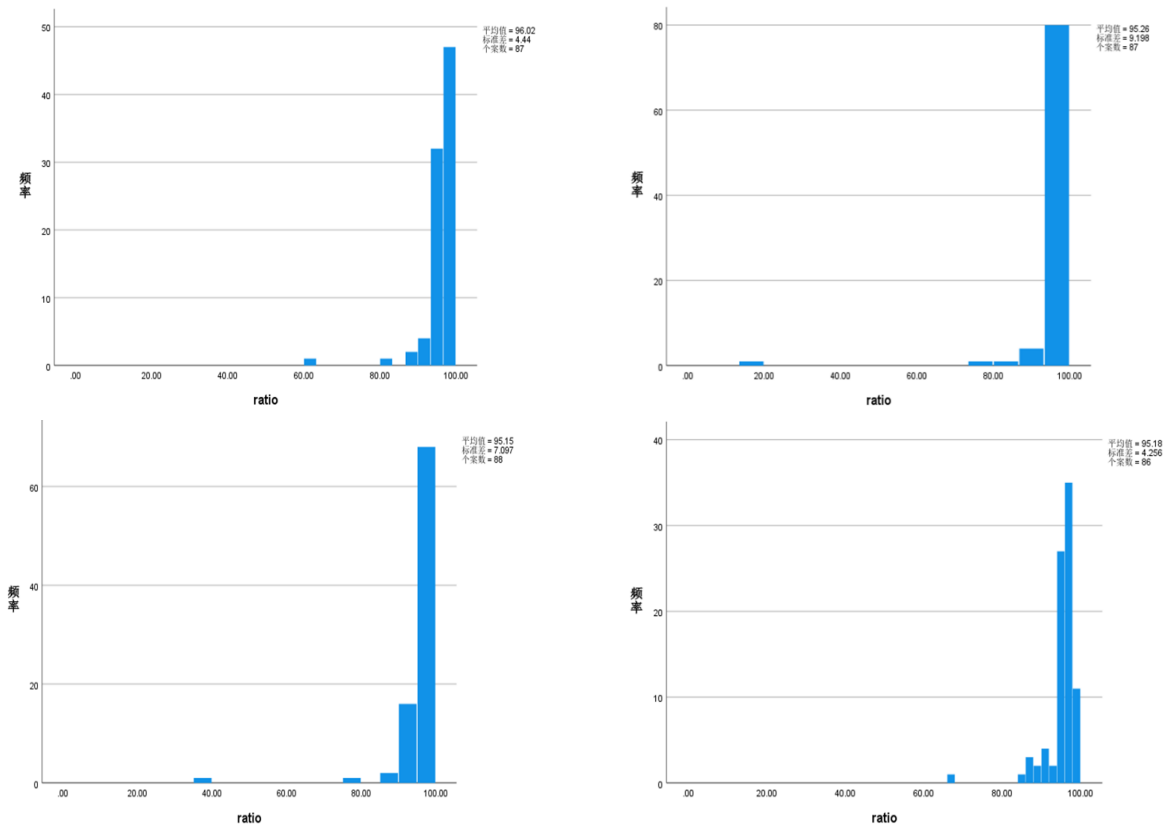
| 数据集 | 实际有效利用率 (%) | 理论极限利用率 (%) |
|-----|-------------|-------------|
| A1  | 96.02       | 99.45       |
| A2  | 94.08       | 99.85       |
| A3  | 95.15       | 99.68       |
| A4  | 94.08       | 99.82       |

由上表可知，实际有效利用率与理论极限利用率相差不大，可认为该算法有效。

2) 基于 SPSS 的排样效果描述性统计分析

利用 SPSS 软件对每一数据集得到的排样结果做描述性统计分析，结果如下：

| 数据集 | 原片数 | 最小值   | 最大值   | 极差    | 均值      | 方差     | 标准偏差    |
|-----|-----|-------|-------|-------|---------|--------|---------|
| A1  | 87  | 61.67 | 99.26 | 37.59 | 96.0245 | 19.710 | 4.43961 |
| A2  | 87  | 14.92 | 99.67 | 84.75 | 95.2579 | 84.601 | 9.19786 |
| A3  | 88  | 35.34 | 99.19 | 63.85 | 95.1466 | 50.364 | 7.09676 |
| A4  | 86  | 66.26 | 98.90 | 32.64 | 95.1783 | 18.115 | 4.25619 |



由 SPSS 输出结果可知，对于四个数据集，该排样方案的利用率均值均在 95%以上，且由极差、方差、标准偏差可知其离散程度较小，即数据较为集中。可认为该算法有效。

## 六、问题二：模型建立与求解

### 6.1 混合整数规划模型建立

假设共有  $n$  组订单  $(O_1, \dots, O_n)$ ，其中含有的产品项为  $(x_1, \dots, x_n)$  个，包含的材料数量为  $Q$ 。

记  $\xi_{st} (1 \leq s \leq n, 1 \leq t \leq x_s)$  为第  $s$  个订单中生产的第  $t$  个产品项，这个产品项的长度、宽度及需求量可表示为  $(l_{st}, w_{st}, d_{st})$ 。当第  $\xi_{st}$  个产品项使用了第  $v$  种材料时， $\mu_{stv}$  取 1，否则取 0。

设所有订单可分为  $m$  个批次  $(b_1, \dots, b_m)$ ，当第  $s$  个订单在第  $u$  个批次里时， $v_{su}$  取 1，否则取 0。

约束条件如下：

- 每份订单当且仅当出现在一个批次中；
- 每个批次可生产的产品项总数不超过限定值，即有上限；
- 每个批次可生产的产品项面积总和不超过限定值，即有上限

设函数  $g$  为每批次所需原片数量的一个函数，即  $g(b_i)$  可表示第  $b_i$  个批次所需原片数量，为达到使板材原片数量最少的目的，可建立以下模型<sup>[11]-[12]</sup>：

$$\min \sum_{u=1}^m g(b_u) \quad (24)$$

s.t.

$$\sum_{s=1}^n v_{su} = 1, 1 \leq u \leq m, \quad (25)$$

$$\sum_{s=1}^n v_{su} x_s \leq IN_{\max}, 1 \leq u \leq m, \quad (26)$$

$$\sum_{s=1}^n v_{su} \sum_{t=1}^{x_s} l_{st} w_{st} d_{st} \leq SQ_{\max}, 1 \leq u \leq m. \quad (27)$$

对目标函数精细处理如下：

$$g(b_i) = \sum_{t=1}^Q h(v_{1u} \xi_{11} \mu_{11t}, \dots, v_{1u} \xi_{1x_1} \mu_{1x_1t}, \dots, v_{nu} \xi_{n1} \mu_{n1t}, \dots, v_{nu} \xi_{nx_n} \mu_{nx_nt}), \quad (28)$$

其中， $h(\xi_1, \dots, \xi_t)$  表示同一批次中同一材质的产品项  $(\xi_1, \dots, \xi_t)$  排样所需板材数量。

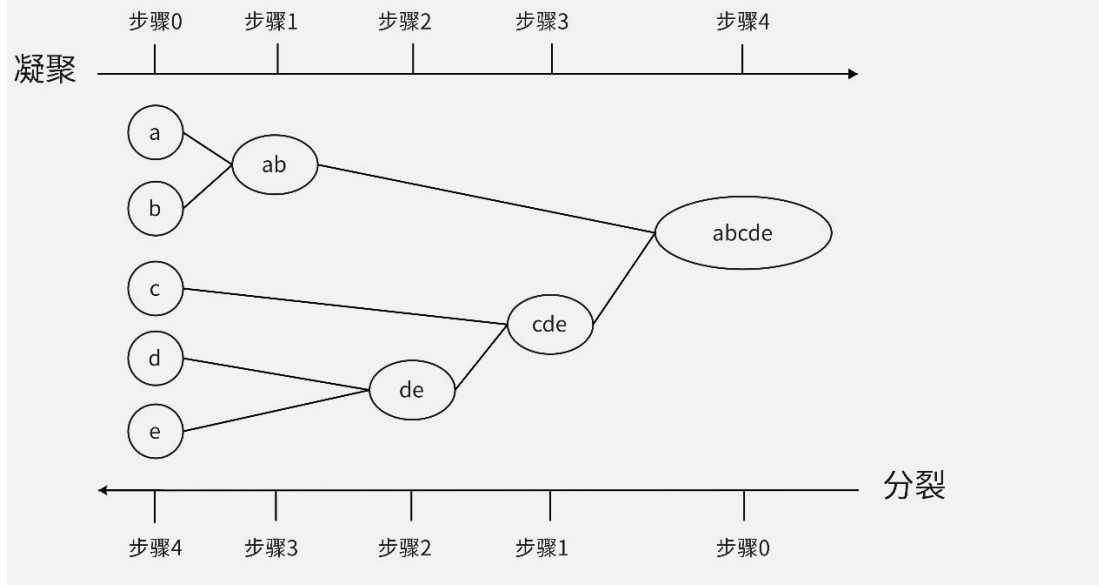
### 6.2 算法设计与建立

#### 6.2.1 订单凝聚层次聚类算法

针对订单组批问题，我们建立了上述的整数规划模型，易见其是一个 NP 完全问题，求其精确解是比较困难的。模型的优化目标是 minimized 板材原片使用数量，若我们在分组批过程中能尽量将使用同种材质的订单放到同一批次中，这样同一批次中的相同材料的产品项种类和数量也越多，也更易排样得到更高的板材使用率，从而也就实现了板材原片使用数

量尽可能小的目标。在此，我们考虑使用启发式算法对模型进行求解。算法基本思想为：通过合理规定的差异度函数计算出订单间的相似程度，并在此基础上对订单进行聚类，最终生成分批后的订单。在本文中，我们提出了一种满足问题各类约束的订单凝聚层次聚类算法对订单进行组批。

层次聚类算法概念图如下所示。



## 1. 订单相似性分析及距离确定

为了能让具有较多相同材质产品项的订单组合在一起，需要建立个订单间之间材质相似性度量方法。本文采用基于杰拉德相似性系数（JSC）来度量两个订单之间材质的相似程度。定义如下：

（1）杰拉德相似系数：两个订单集合 A 和 B 的材质种类交集的元素个数在 A 和 B 的并集的元素个数占比，称之为两个订单集合的杰拉德相似系数，计算表达式如下：

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}. \quad (29)$$

（2）杰拉德距离：杰拉德距离用两个订单集合 A 和 B 中不同材质的元素个数与所有元素的比例来度量两个集合的区分度，即距离，计算表达式如下：

$$J_\delta(A, B) = 1 - J(A, B) = \frac{|A \cup B| - |A \cap B|}{|A \cup B|}. \quad (30)$$

在层次聚类算法中，聚类距离间距的计算方法主要有最短距离法、最长距离法、中间距离法、重心法、类平均距离法这五种方法。本文中关于订单组批的类间距离采用类平均距离法，类平均距离法是计算两个簇之间各订单两两之间距离，将所有距离的均值作为两个订单簇之间的距离。订单簇 I 和 J 之间的距离计算公式如下：

$$D_{IJ} = \sqrt{\frac{1}{n_I n_J} \sum_{i \in I, j \in J} d_{ij}^2}, \quad (31)$$

其中  $d_{ij}^2$  为订单簇 I 中任一订单 i 和订单簇 J 任一订单 j 之间的欧式距离平方。

## 2. 订单凝聚层次聚类算法

订单在进行层次聚类时，我们可以按其创建聚类订单树是采用“自底往上”的方式，故将其分为凝聚层次聚类。在本文算法构建中，首先将每个订单作为一个簇，将最近的一对簇进行合并，并重复迭代此过程，直到所有的簇都不能合并迭代终止。通过这种方式可以创建一颗有层次的嵌套聚类树，即通过计算每一个类别的订单之间的距离来确定它们之间的相似程度，距离越小，相似程度越高，然后按距离准则逐步合并，减少类数。算法流程如下：

**首先设置算法终止条件：**

- 设置类间距离阈值  $F$ ，当  $L(n)$  的最小分量超过给定值  $F$  是，算法终止；
- 当所有类合并后都不满足满足约束条件时，停止合并。

**Step1.** 将  $N$  个初始状态下的订单单独组成一类，即建立  $N$  类： $K_1(0), K_2(0), \dots, K_N(0)$ ，其中 0 表示初始状态，进而计算各类之间即各订单之间的距离，即得到一个  $N$  维的距离矩阵。

**Step2.** 计算各类（订单簇）之间的类平均距离矩阵  $L(n)$ ，其中  $n$  为逐次聚类合并的次数，并找出  $L(n)$  最小的元素及其对应的两个类  $K_x(n), K_y(n)$ 。

**Step3.** 将  $K_x(n)$  和  $K_y(n)$  这两类尝试合并成  $K_0(n)$ ，判断合成后类  $K_0(n)$  中的所有订单中产品项总数、产品项面积之和是否满足约束条件，如果不满足，这返回上一步，找出  $L(n)$  次小的元素所对应的类进行合并检验，若满足约束条件，则进行合并，并由此建立新的分类： $K_1(n+1), K_2(n+1)$  等等。

**Step4.** 计算合并后新类之间的距离矩阵，得  $L(n+1)$ 。

**Step5.** 转至 **step2**，重复计算并进行合并。

算法结束后，即得聚类结果。

### 6.3 模型求解

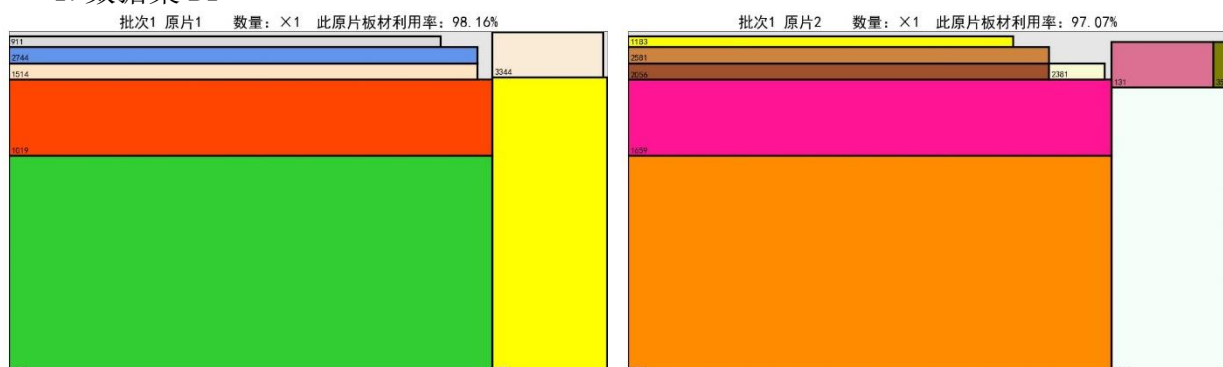
基于**层次聚类算法**对模型进行订单组批，并将所得结果中同一批次按不同材质依次按照上文第五部分的 TUSP 与 SVC 算法进行排样，最终可得排样方案。

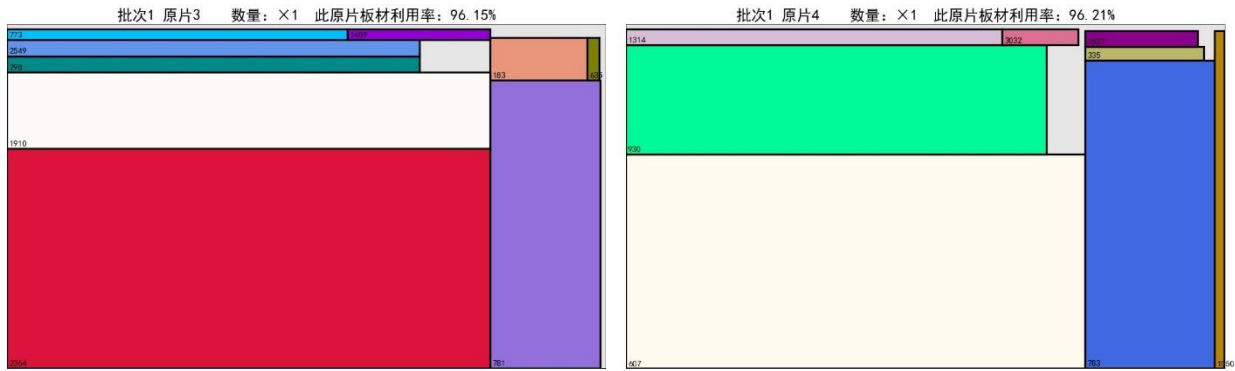
### 6.4 求解结果与分析

#### 6.4.1 对于数据集 B1、B2、B3、B4 的排样方案效果图

由于订单数量与产品项种类、数量众多，无法一一展示按算法所得排样效果图，下面截取部分结果进行展示（完整排样方案效果图见附件）

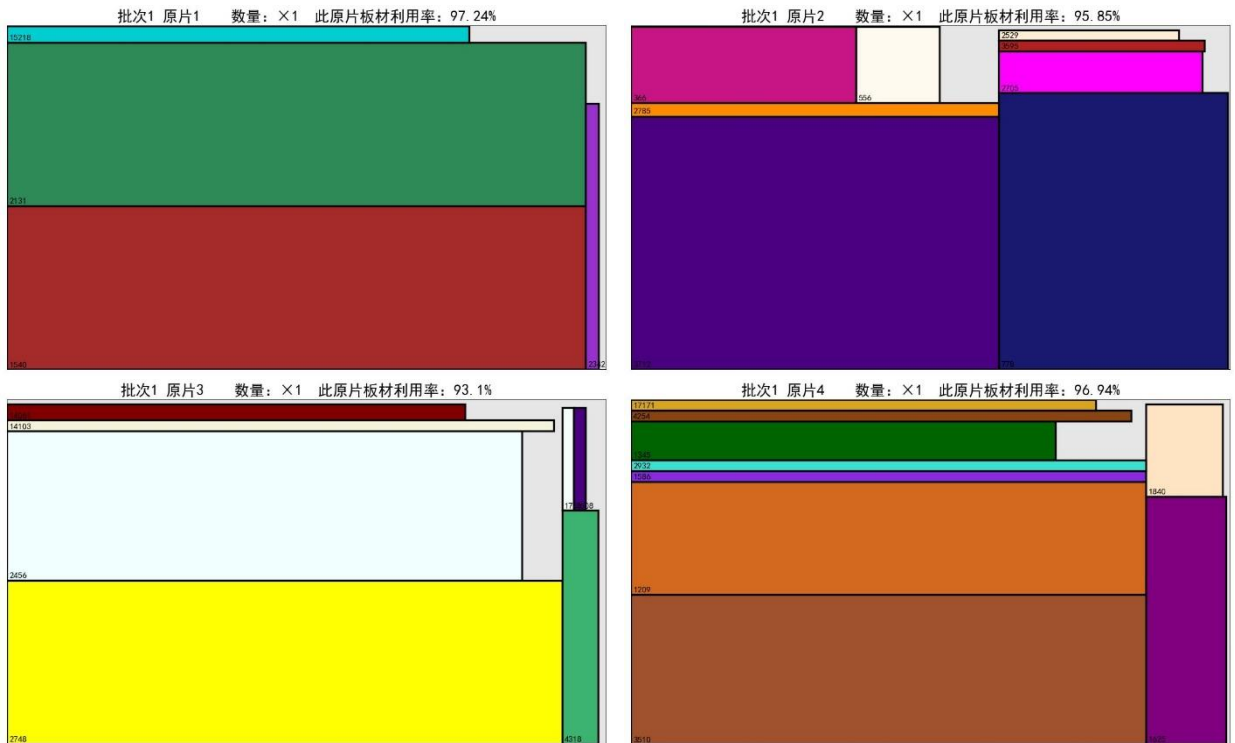
##### 1. 数据集 B1





| 批次序号 | 原片材质      | 原片序号 | 产品id | 产品 x 坐标 | 产品 y 坐标 | 产品 x 方向长度 | 产品 y 方向长度 |
|------|-----------|------|------|---------|---------|-----------|-----------|
| 1    | YW1-0218S | 1    | 857  | 1968    | 0       | 468       | 1058      |
| 1    | YW1-0218S | 1    | 911  | 0       | 1166    | 1758      | 38        |
| 1    | YW1-0218S | 1    | 1019 | 0       | 780     | 1968      | 270       |
| 1    | YW1-0218S | 1    | 1156 | 0       | 0       | 1968      | 780       |
| 1    | YW1-0218S | 1    | 1514 | 0       | 1050    | 1907      | 58        |
| 1    | YW1-0218S | 1    | 2744 | 0       | 1108    | 1907      | 58        |
| 1    | YW1-0218S | 1    | 3344 | 1968    | 1058    | 451       | 161       |

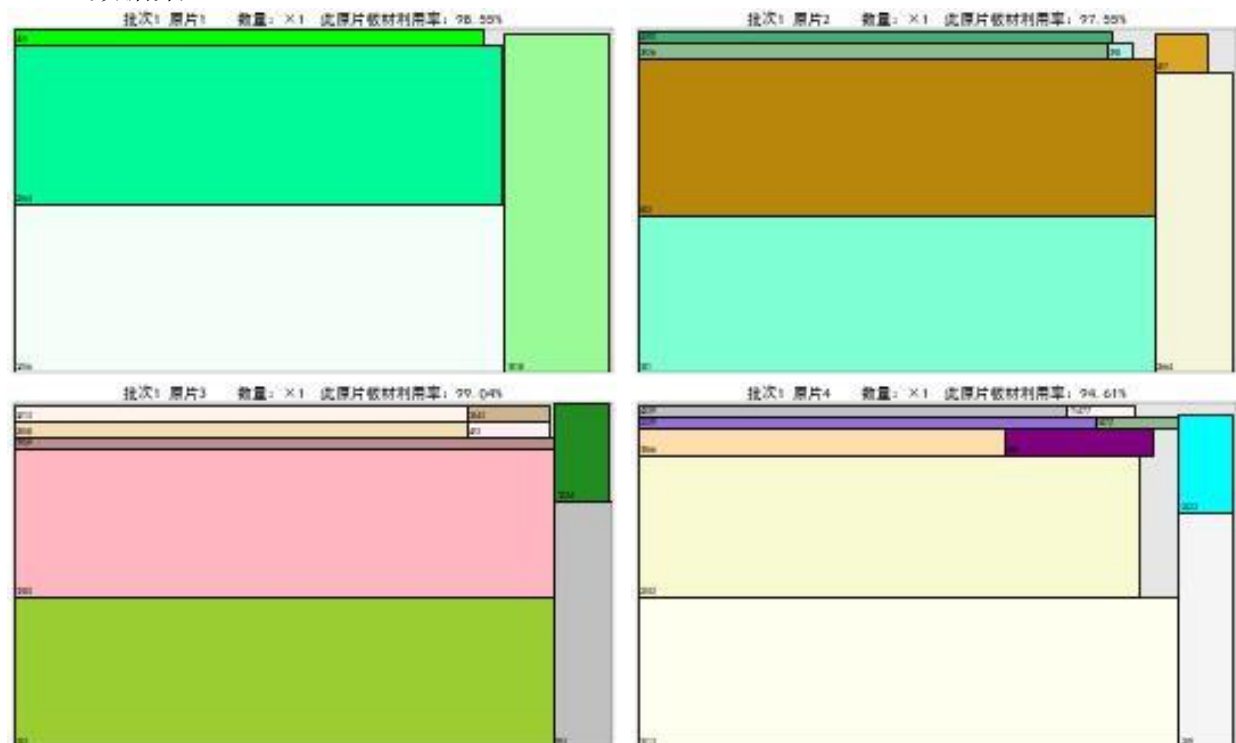
## 2. 数据集 B2



| 批次序号 | 原片材质 | 原片序号 | 产品id | 产品 x 坐标 | 产品 y 坐标 | 产品 x 方向长度 | 产品 y 方向长度 |
|------|------|------|------|---------|---------|-----------|-----------|
|------|------|------|------|---------|---------|-----------|-----------|

|   |           |   |       |      |      |      |     |
|---|-----------|---|-------|------|------|------|-----|
| 1 | YSH-0218S | 1 | 1540  | 0    | 0    | 2358 | 580 |
| 1 | YSH-0218S | 1 | 2131  | 0    | 580  | 2358 | 580 |
| 1 | YSH-0218S | 1 | 2342  | 2358 | 0    | 53   | 944 |
| 1 | YSH-0218S | 1 | 15218 | 0    | 1160 | 1883 | 58  |
| 1 | YSH-0218S | 2 | 366   | 0    | 946  | 917  | 270 |
| 1 | YSH-0218S | 2 | 556   | 917  | 946  | 341  | 270 |
| 1 | YSH-0218S | 2 | 778   | 1498 | 0    | 932  | 981 |

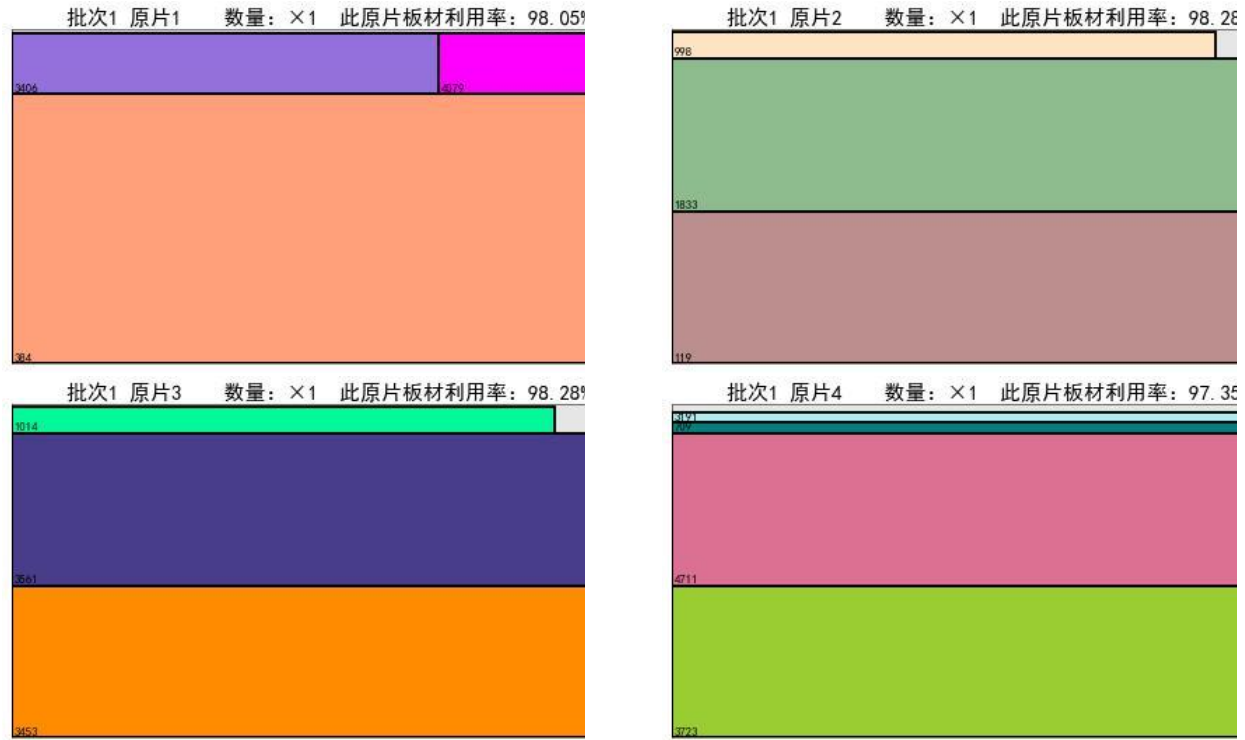
### 3. 数据集 B3



| 批次序号 | 原片材质      | 原片序号 | 产品id | 产品 x 坐标 | 产品 y 坐标 | 产品 x 方向长度 | 产品 y 方向长度 |
|------|-----------|------|------|---------|---------|-----------|-----------|
| 1    | QKQ-0218S | 1    | 455  | 0       | 1162    | 1904      | 58        |
| 1    | QKQ-0218S | 1    | 1018 | 1995    | 0       | 425.5     | 1203      |
| 1    | QKQ-0218S | 1    | 2564 | 0       | 598     | 1983      | 564       |
| 1    | QKQ-0218S | 1    | 3256 | 0       | 0       | 1995      | 598       |
| 1    | QKQ-0218S | 2    | 381  | 0       | 0       | 2098      | 558       |
| 1    | QKQ-0218S | 2    | 398  | 1904    | 1116    | 102       | 58        |
| 1    | QKQ-0218S | 2    | 427  | 2098    | 1069    | 218       | 133       |

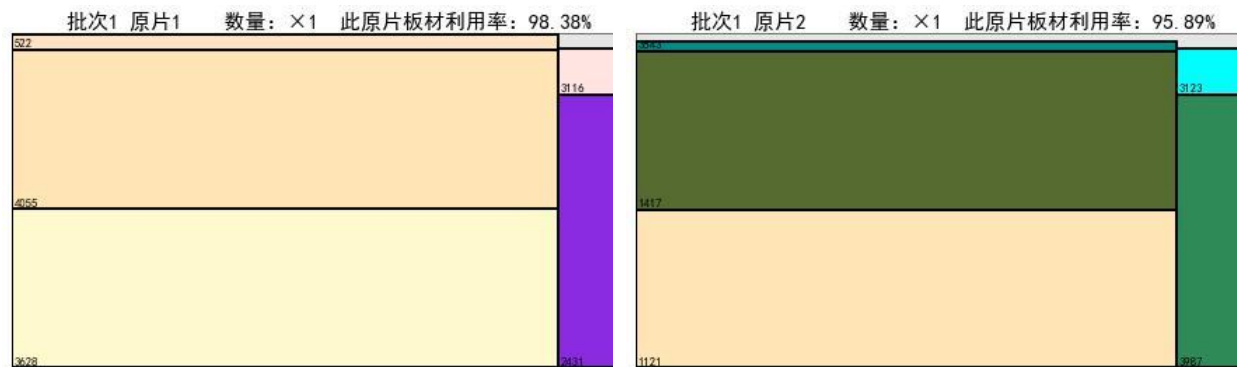


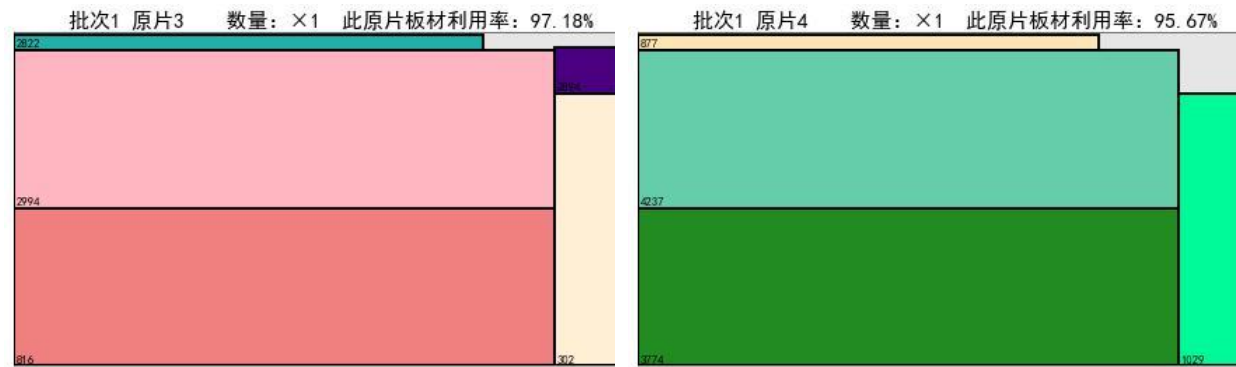
#### 4. 数据集 B4



| 批次<br>序号 | 原片材质      | 原片<br>序号 | 产品<br>id | 产品 x 坐标 | 产品 y 坐标 | 产品 x<br>方向长<br>度 | 产品 y<br>方向长<br>度 |
|----------|-----------|----------|----------|---------|---------|------------------|------------------|
| 1        | YTM-0218S | 1        | 384      | 0       | 0       | 2375             | 985              |
| 1        | YTM-0218S | 1        | 1523     | 2375    | 881     | 58               | 318              |
| 1        | YTM-0218S | 1        | 2173     | 2375    | 0       | 58               | 881              |
| 1        | YTM-0218S | 1        | 3406     | 0       | 985     | 1643             | 223.5            |
| 1        | YTM-0218S | 1        | 4079     | 1643    | 985     | 638              | 223.5            |
| 1        | YTM-0218S | 2        | 119      | 0       | 0       | 2375             | 558              |
| 1        | YTM-0218S | 2        | 449      | 2375    | 0       | 58               | 821              |

#### 5. 数据集 B5





| 批次序号 | 原片材质       | 原片序号 | 产品id | 产品 x 坐标 | 产品 y 坐标 | 产品 x 方向长度 | 产品 y 方向长度 |
|------|------------|------|------|---------|---------|-----------|-----------|
| 1    | YW10-0218S | 1    | 522  | 0       | 1160    | 2098      | 58        |
| 1    | YW10-0218S | 1    | 2431 | 2098    | 0       | 329       | 998       |
| 1    | YW10-0218S | 1    | 3116 | 2098    | 998     | 269       | 167       |
| 1    | YW10-0218S | 1    | 3628 | 0       | 0       | 2098      | 580       |
| 1    | YW10-0218S | 1    | 4055 | 0       | 580     | 2098      | 580       |
| 1    | YW10-0218S | 2    | 1121 | 0       | 0       | 2078      | 578       |
| 1    | YW10-0218S | 2    | 1417 | 0       | 578     | 2078      | 578       |

#### 6.4.2 结果分析

利用 matlab 软件进行编程求解，对于每个数据集，其计算结果如下所示：

| 数据集 | 批次数 | 总原片数 | 原片使用率           |
|-----|-----|------|-----------------|
| B1  | 164 | 3528 | <b>84.4190%</b> |
| B2  | 164 | 2326 | <b>82.8402%</b> |
| B3  | 177 | 2355 | <b>82.1122%</b> |
| B4  | 162 | 2433 | <b>83.4418%</b> |
| B5  | 220 | 3700 | <b>83.5051%</b> |

由上表可以看出，对于五个数据集，原片的利用率均值均在 80% 以上，可以看出我们的算法效果是较好的。

## 七、模型分析与评价

### 7.1 算法复杂度分析

对于 3 阶段二维矩形排样问题的算法设计，我们是使用基于 HUSP 算法和价值修正的顺序启发式算法来寻找满足约束条件和优化目标的解，其中 HUSP 算法主要包括三个层级，对应着 3 阶段切样方式。第一个层级是产品项生成栈，需要遍历所有的个栈；第二个层级是栈生成条带，需要遍历所有的条带，第三个层级是条带生成原片，需要遍历所有的原片，因此算法的复杂度为  $O(n^3)$

---

## 7.2 模型优缺点分析

### 7.2.1 模型优点分析

1、本文考虑的三阶段排样问题，考虑了较多约束条件，如齐头切和产品项可旋转等，这保证了本文所建立的模型和算法具有较高的灵活性，使用范围较广。

2、本文针对 3 阶段二维矩形排样问题建立了混合整数规划模型，并结合优化目标和约束条件，设计了基于 HUSP 算法，使用顺序价值修正的启发式算法，算法的求解得到的原片利用率是较好的。

### 7.2.2 模型缺点分析

1、本文所涉及的算法为启发式算法，无法保证算法求解的最优性。

2、对于问题二的处理，我们并没有对订单组批和排样问题进行协同优化，这导致了我们的问题二求解的结果中原片利用率比问题一平均低了 10%。

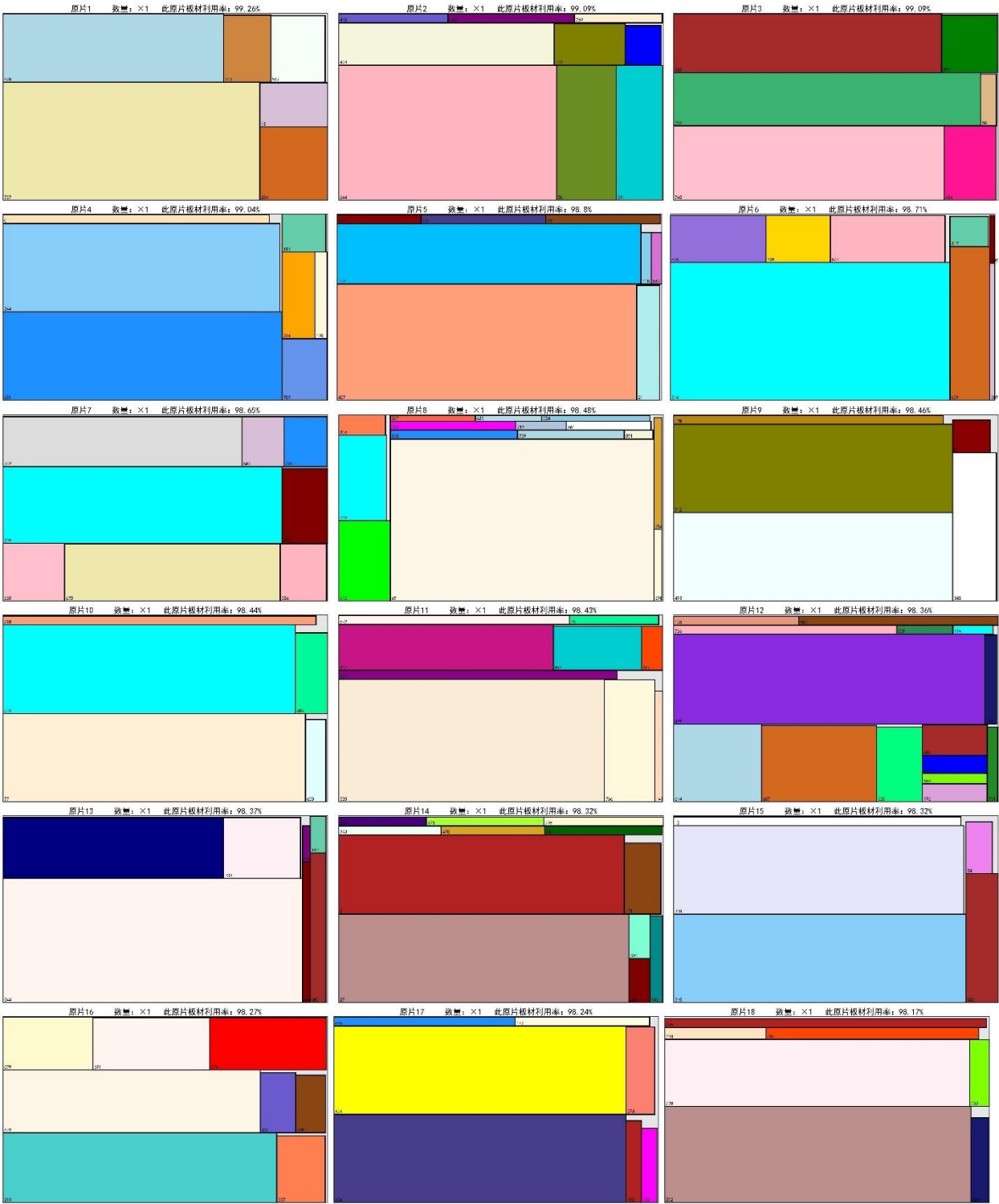
## 八、参考文献

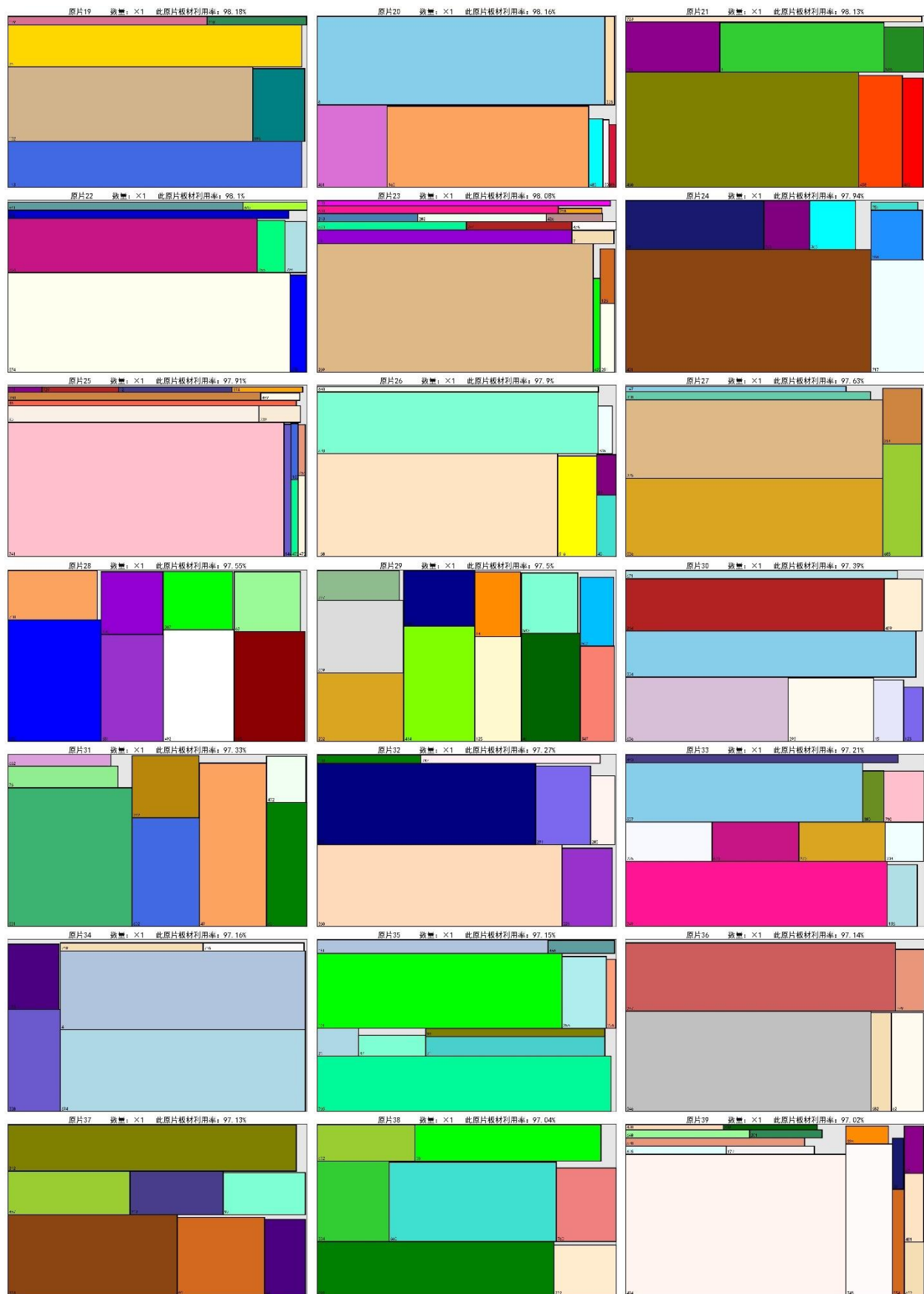
- [1]Chen Q L, Cui Y D, Chen Y, Sequential value correction heuristic for the two-dimensional cutting stock problem with 3-staged homogenous patterns, Optimization Methods and Software, 31(1): 68- 87, 2016.
- [2]Silva E, Alvelos F, Valério de Carvalho J M. An integer programming model for two-and three-stage two-dimensional cutting stock problems , European Journal of Operational Research, 205(3): 699-708, 2010.
- [3]Cui Y D, Huang B X, Reducing the number of cuts in generating three-staged cutting patterns, European Journal of Operational Research, 218: 358-365, 2012.
- [4]Puchinger J, Raidl G R, Models and algorithms for three-stage two-dimensional bin packing, European Journal of Operational Research, 183(3): 1304-1327, 2007.
- [5]Haessler R W, Controlling Cutting Pattern Changes in One-Dimensional Trim Problem, Operations Research, 23(3):483-493, 1975.
- [6]Bclov G, Scheithauer G, Setup and open-stacks minimization in one-dimensional stock cutting, INFORMS Journal on Computing, 19(1):27-35, 2007.
- [7]孔令熠，基于普通条带的二维多阶段排样算法，广西大学，2014.
- [8]李立平，二维三阶段排样算法研究，广西大学，2016.
- [9]陈秋莲，王成栋，二维剪切排样的束搜索启发式算法，计算机工程与应用, 53(09):236-239+257, 2017.
- [10]扈少华，潘立武，管卫利，复合条带三阶段排样方式的生成算法，锻压技术, 41(11):149-152, 2016.
- [11]陈炫锐，陈庆新，毛宁，板式家具企业的订单组批问题研究，工业工程, 22(02):96-102, 2019.
- [12]张浩，面向板式产品定制生产的组批与排样协同优化方法，广东工业大学，2019.

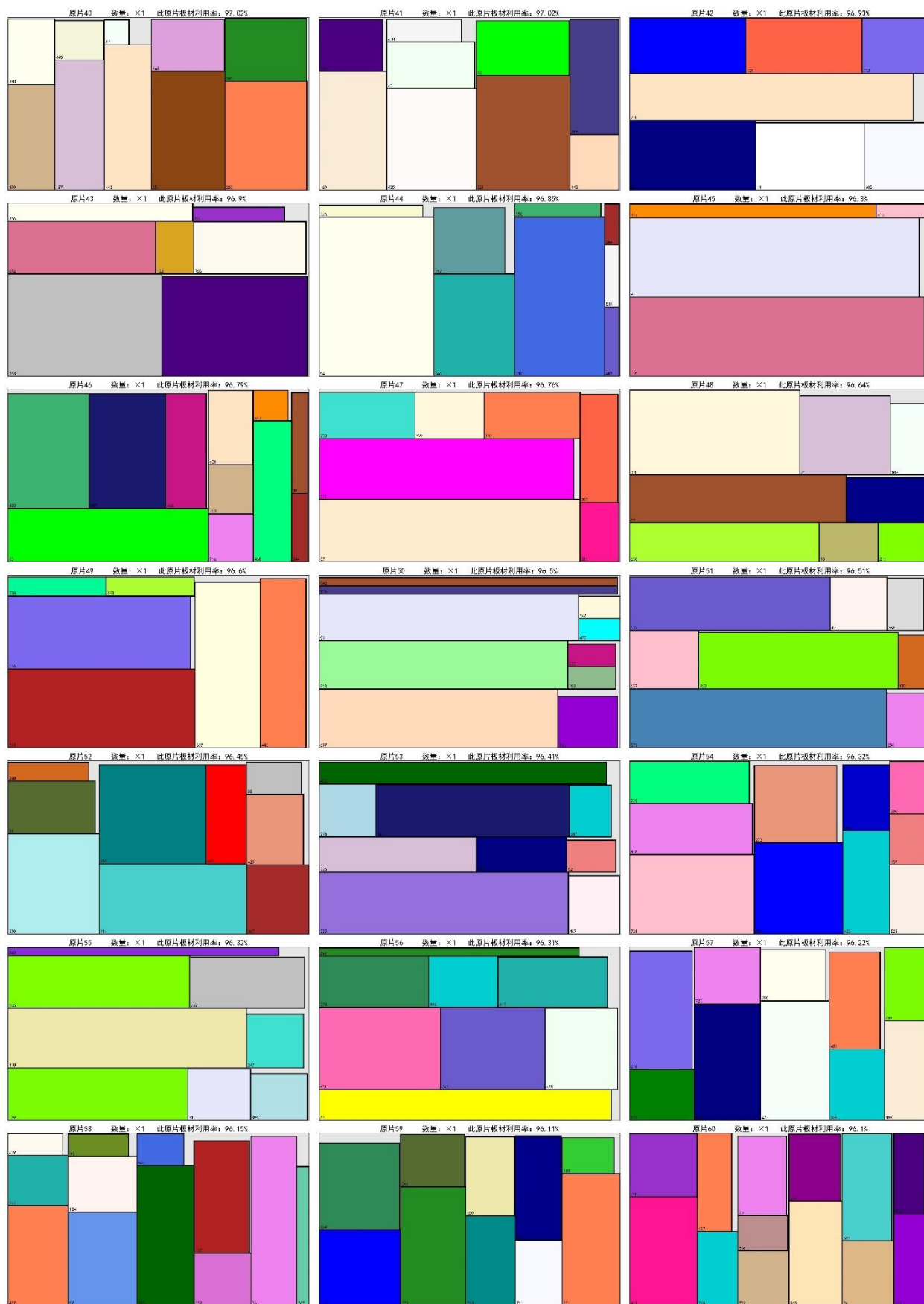
## 九、 附录

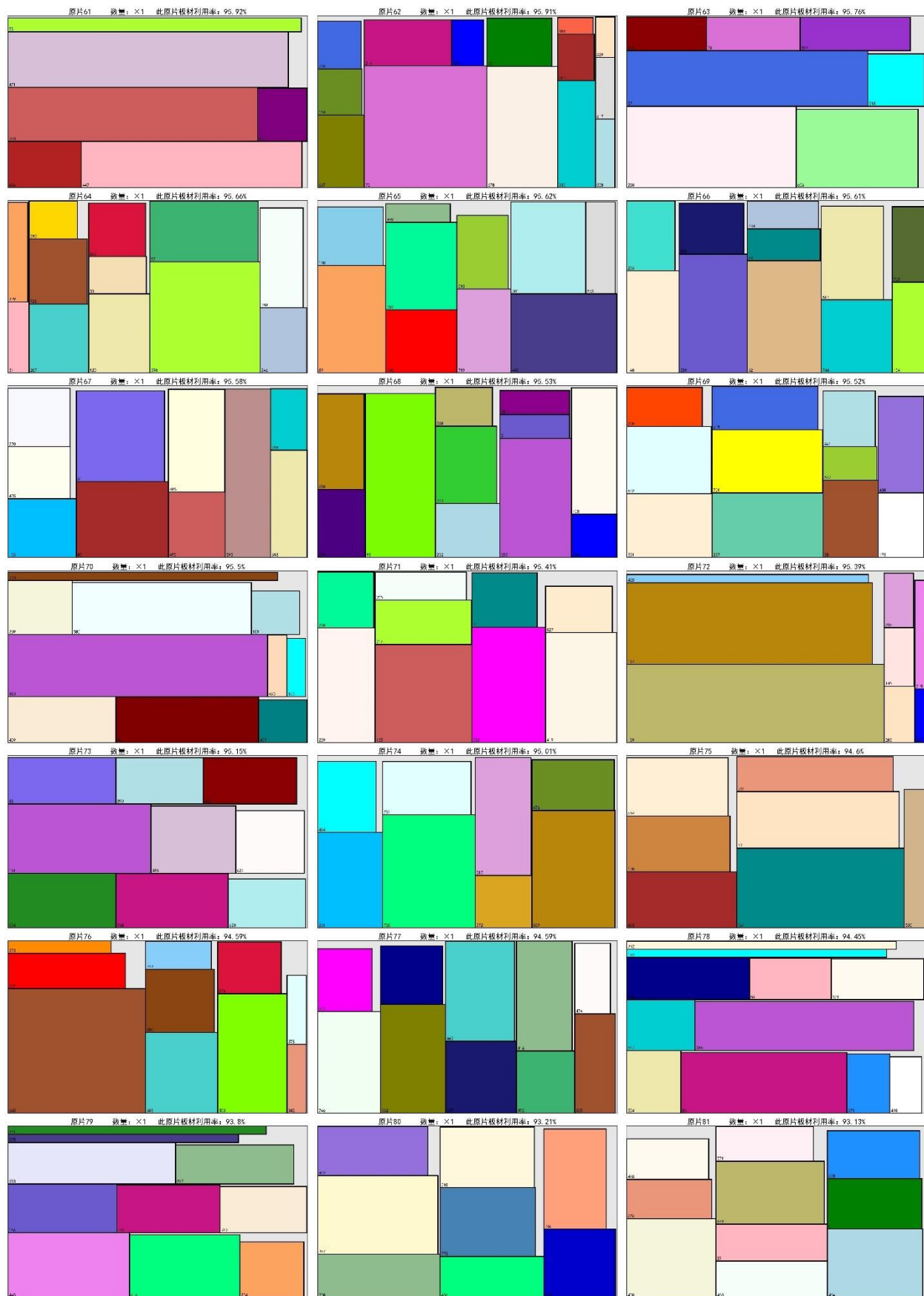
### 9.1 针对数据集 A1、A2、A3、A4 的完整排样方案效果图

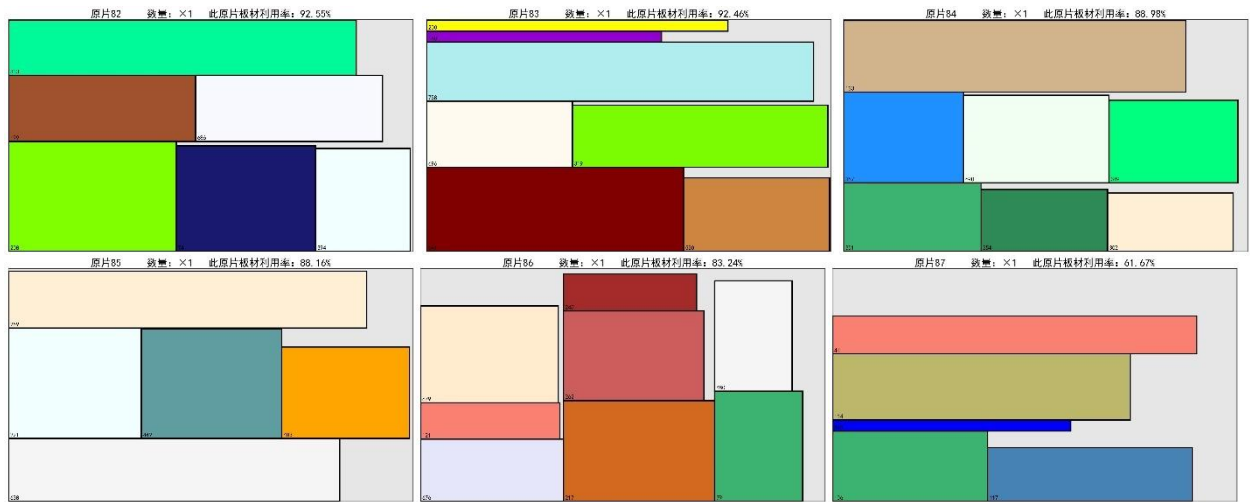
#### 1) 数据集 A1



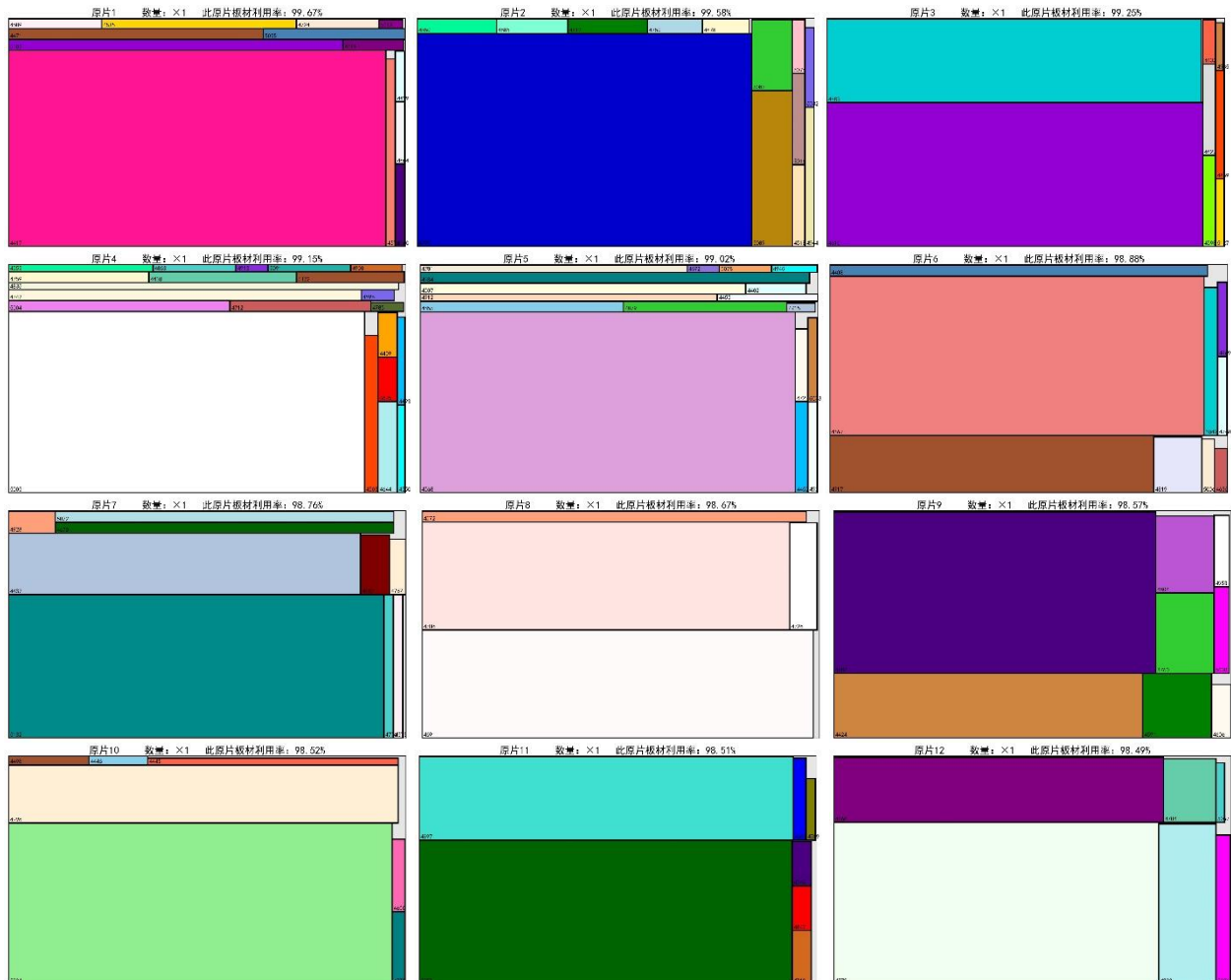




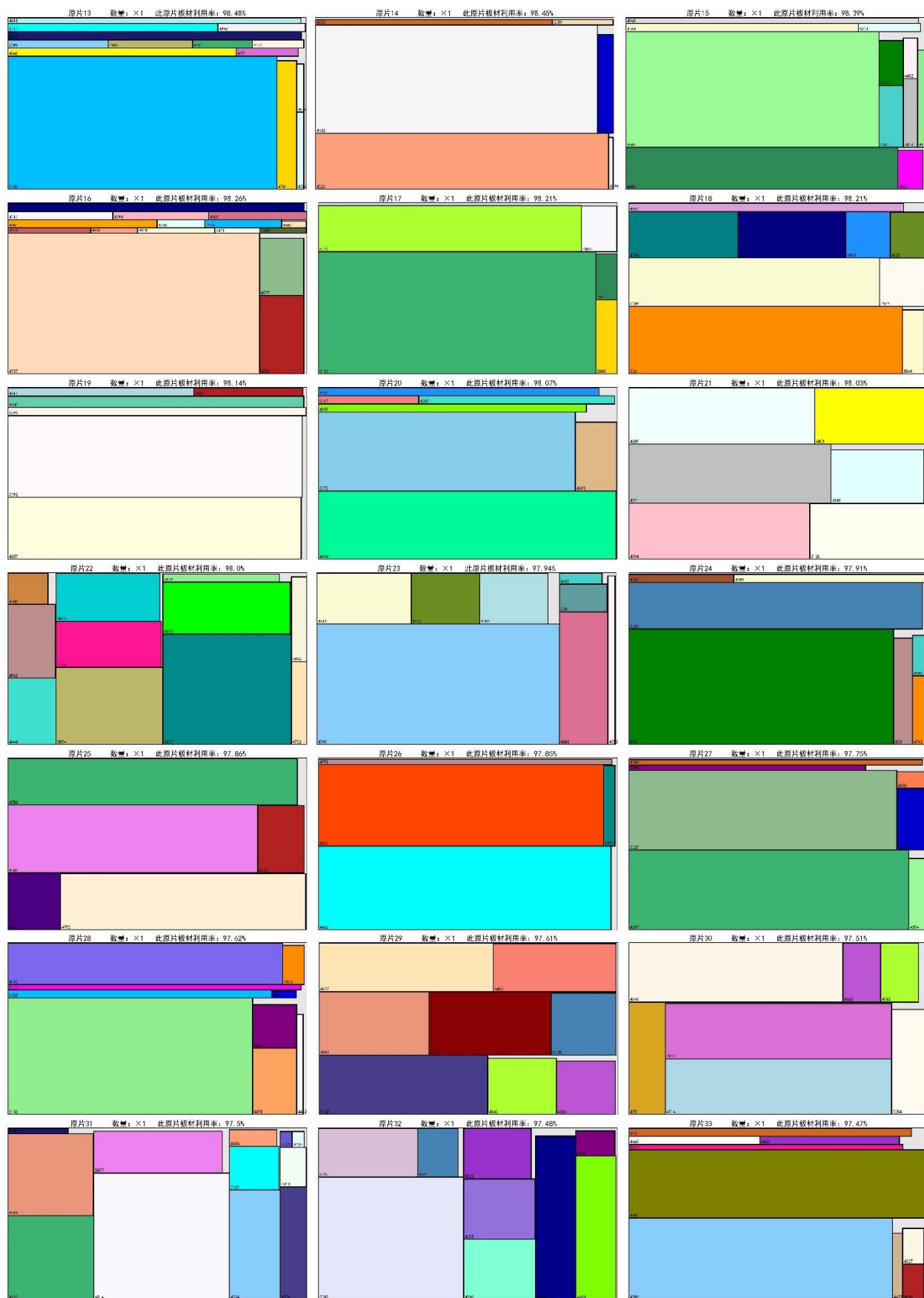


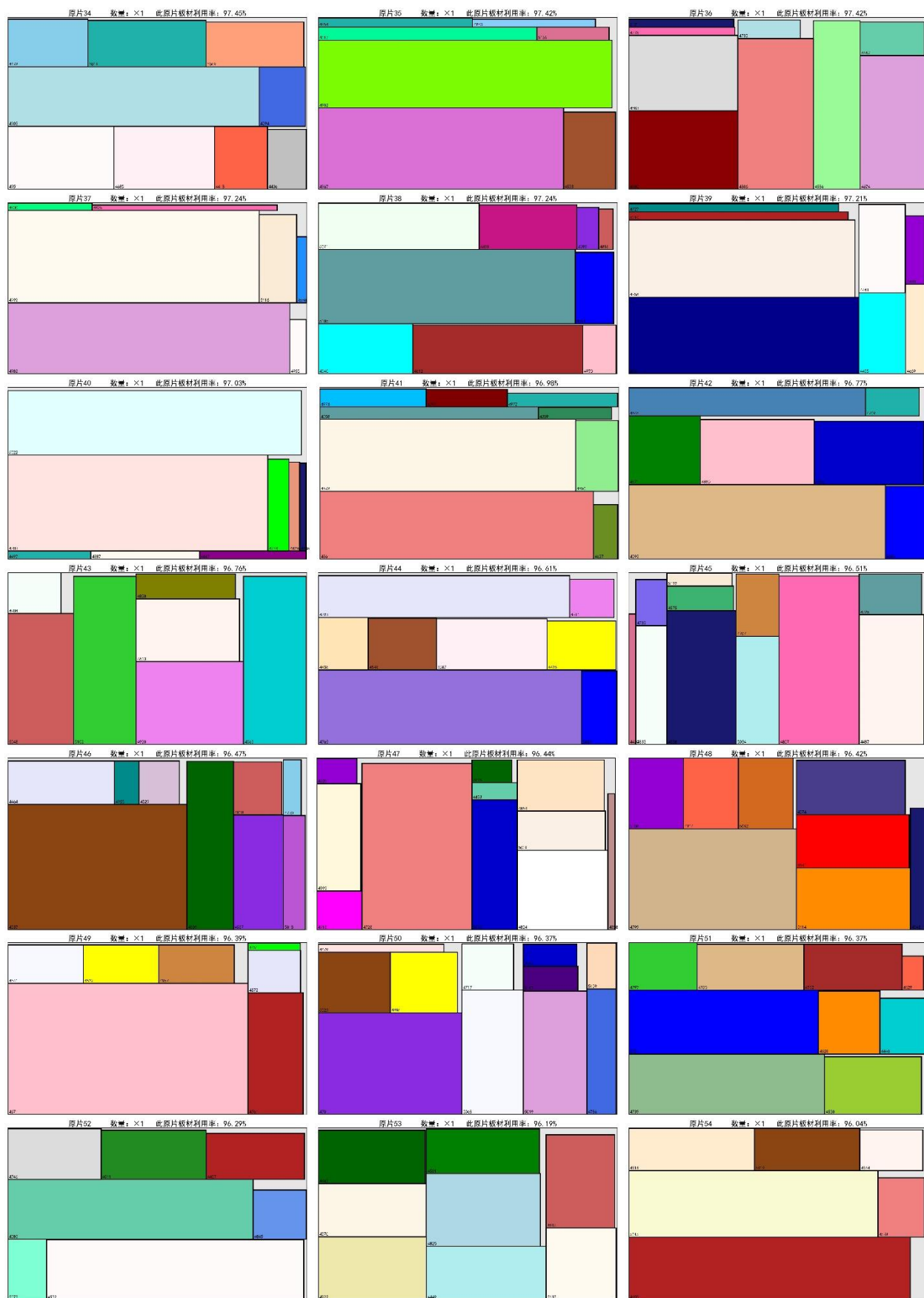


## 2) 数据集 A2

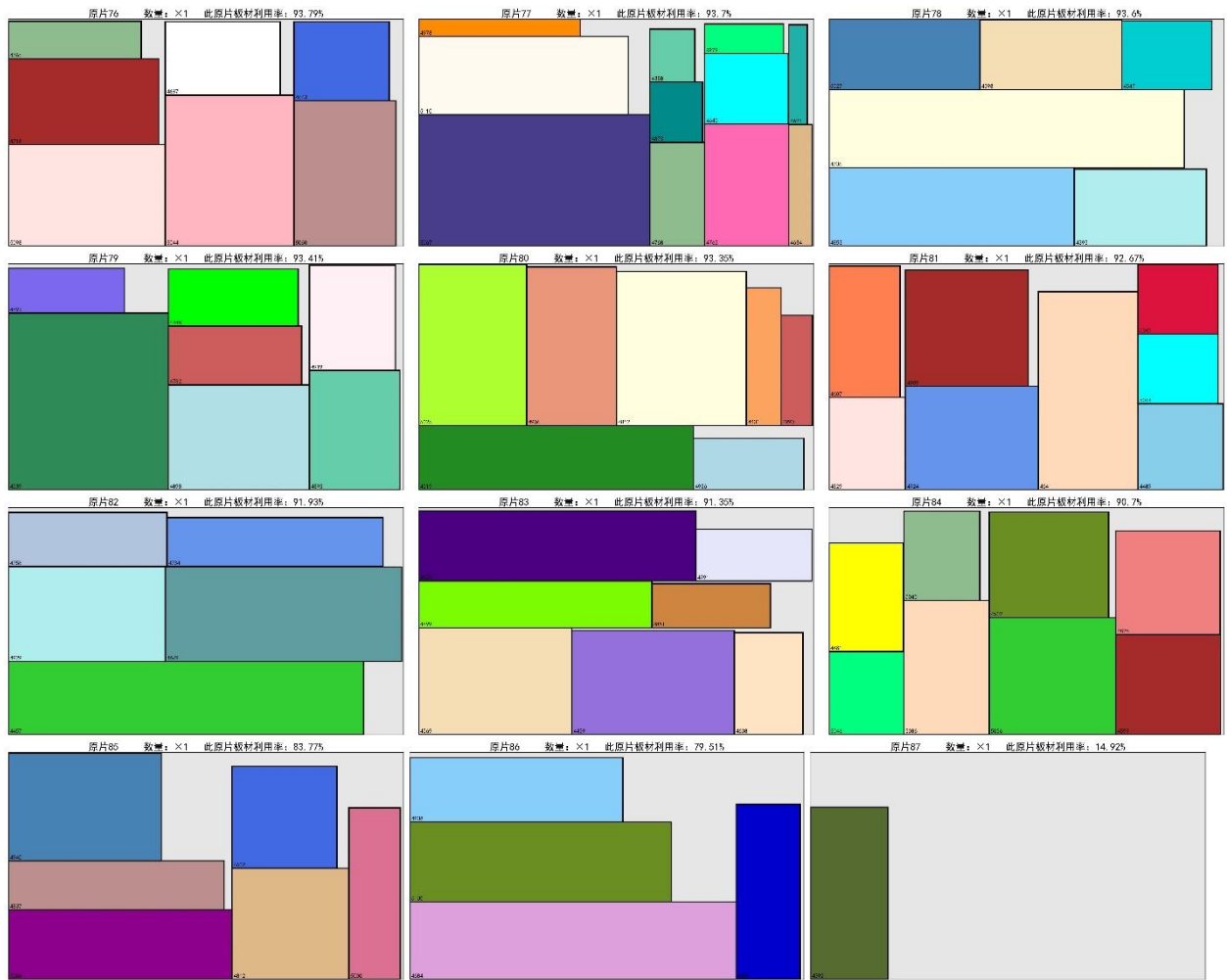




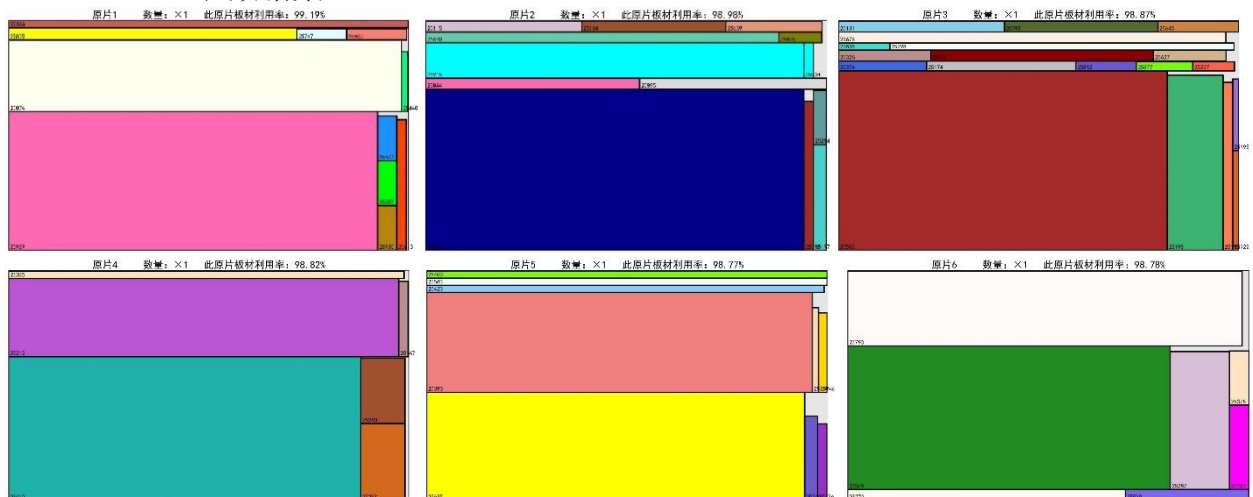


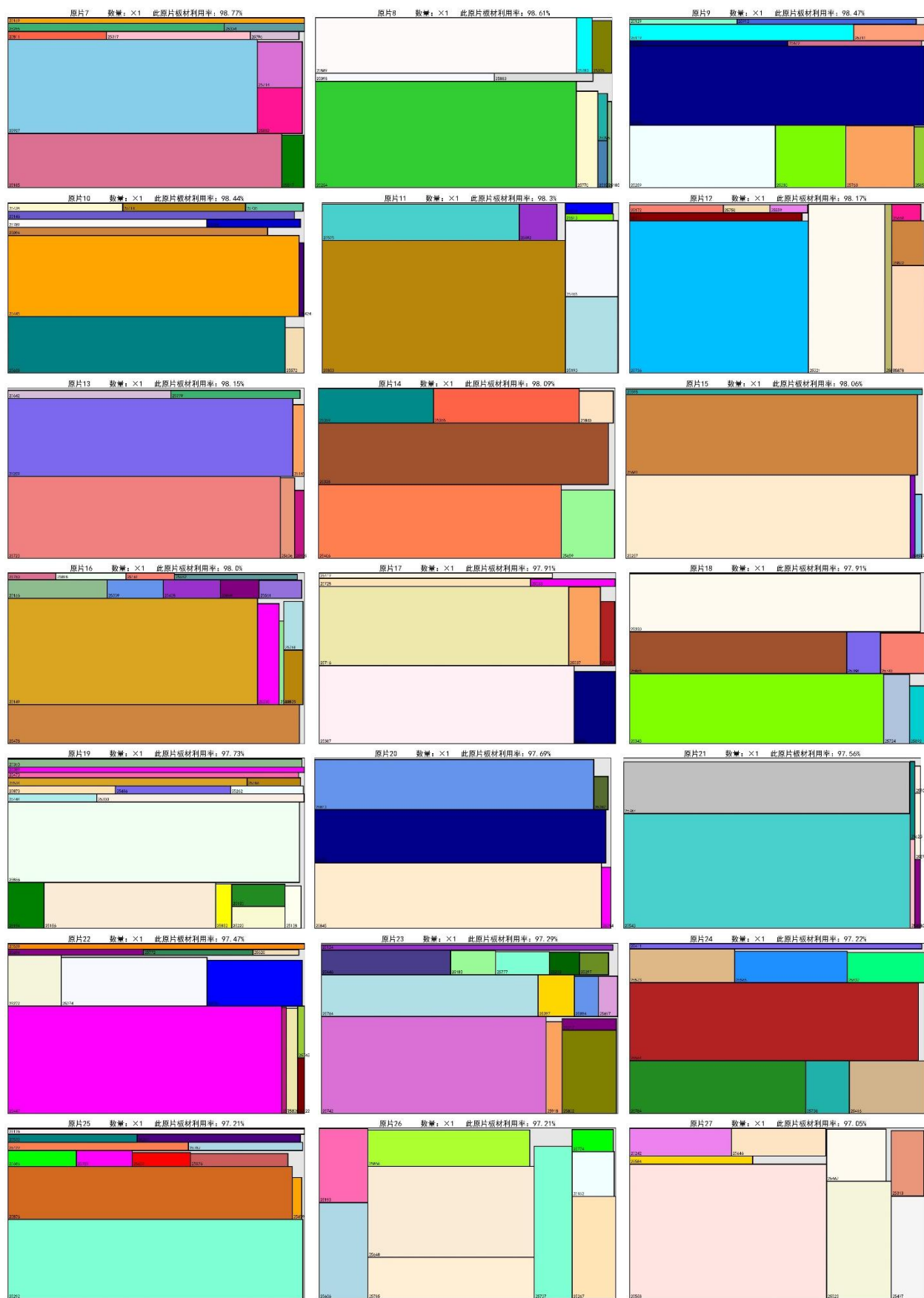


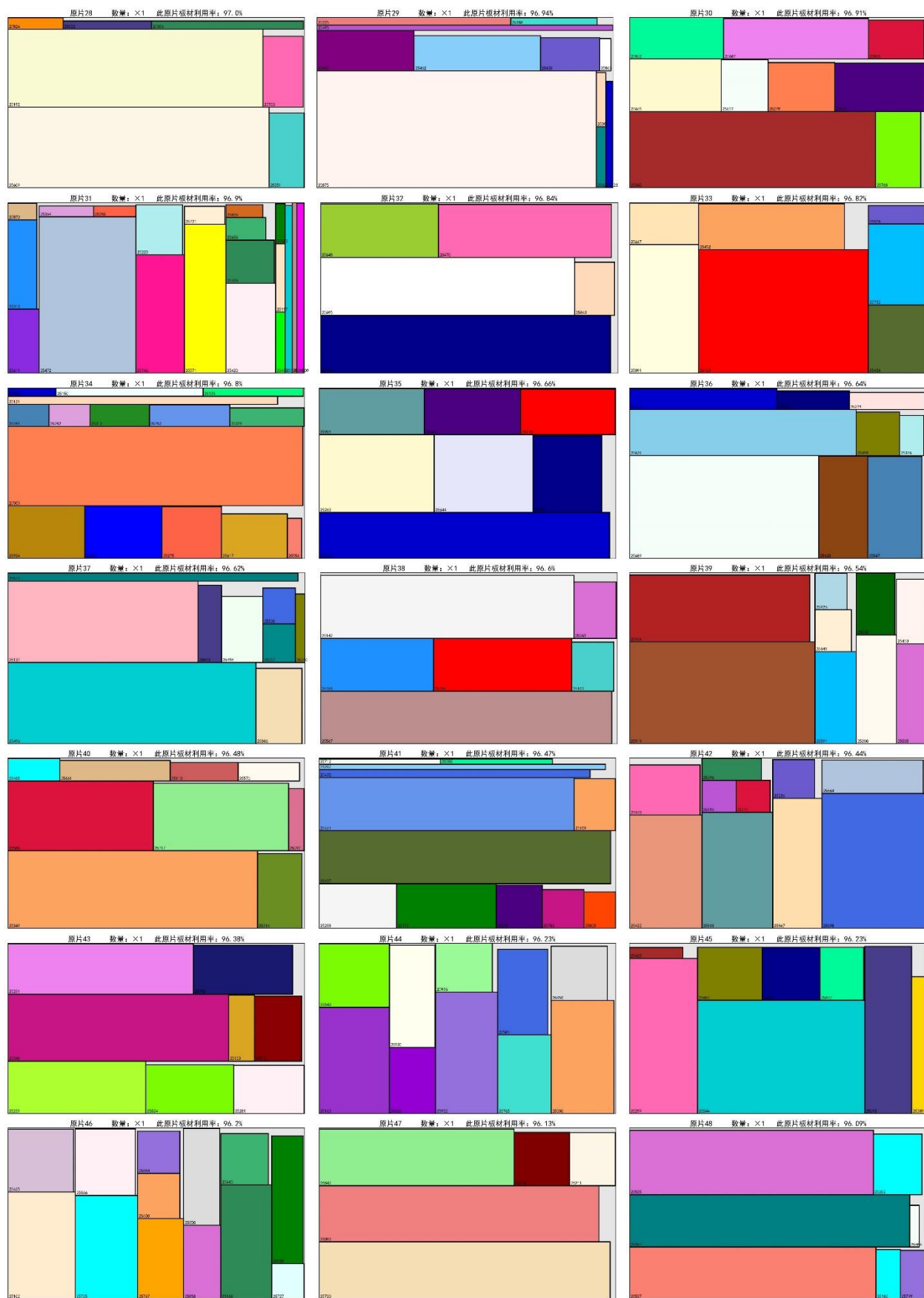


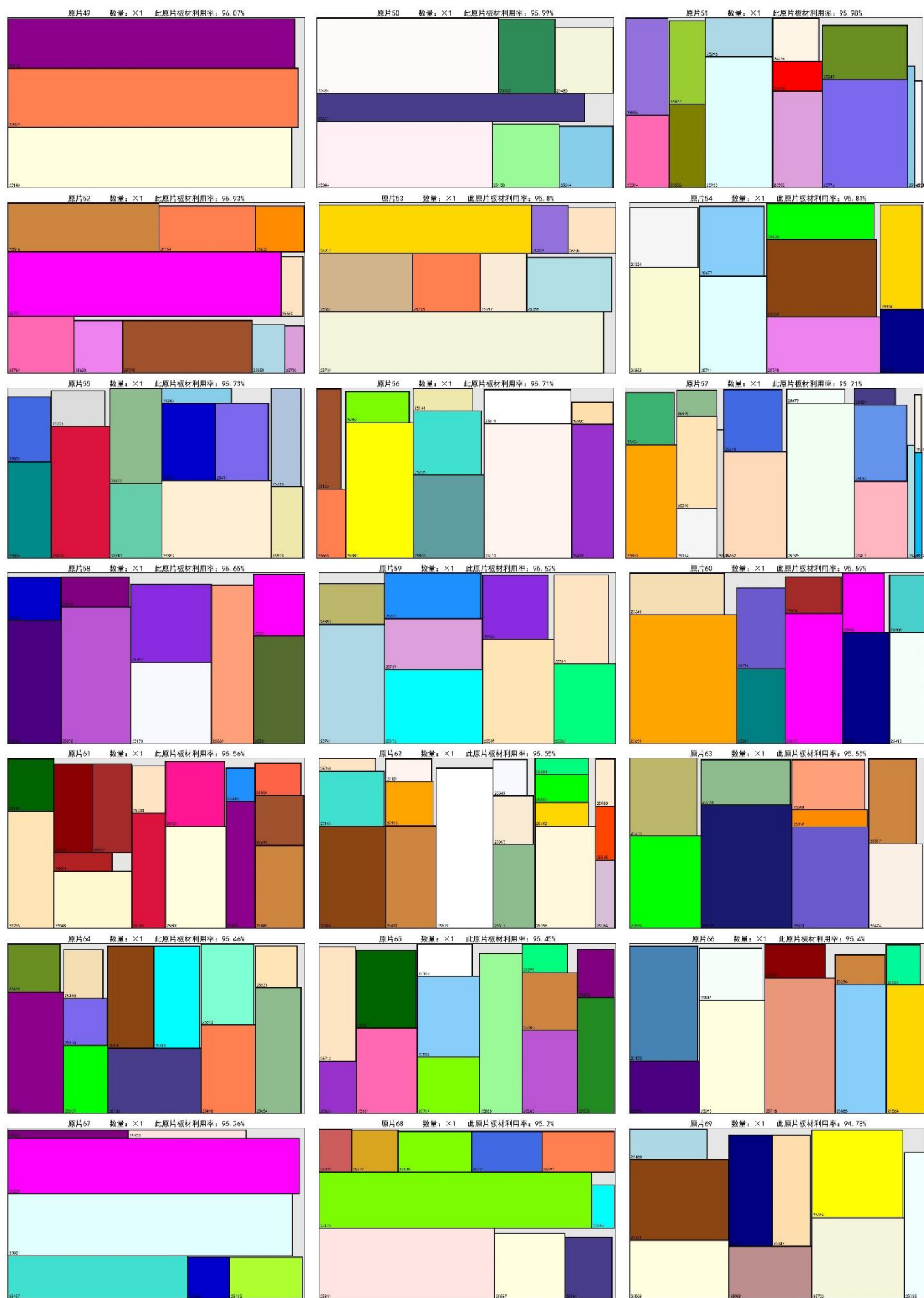


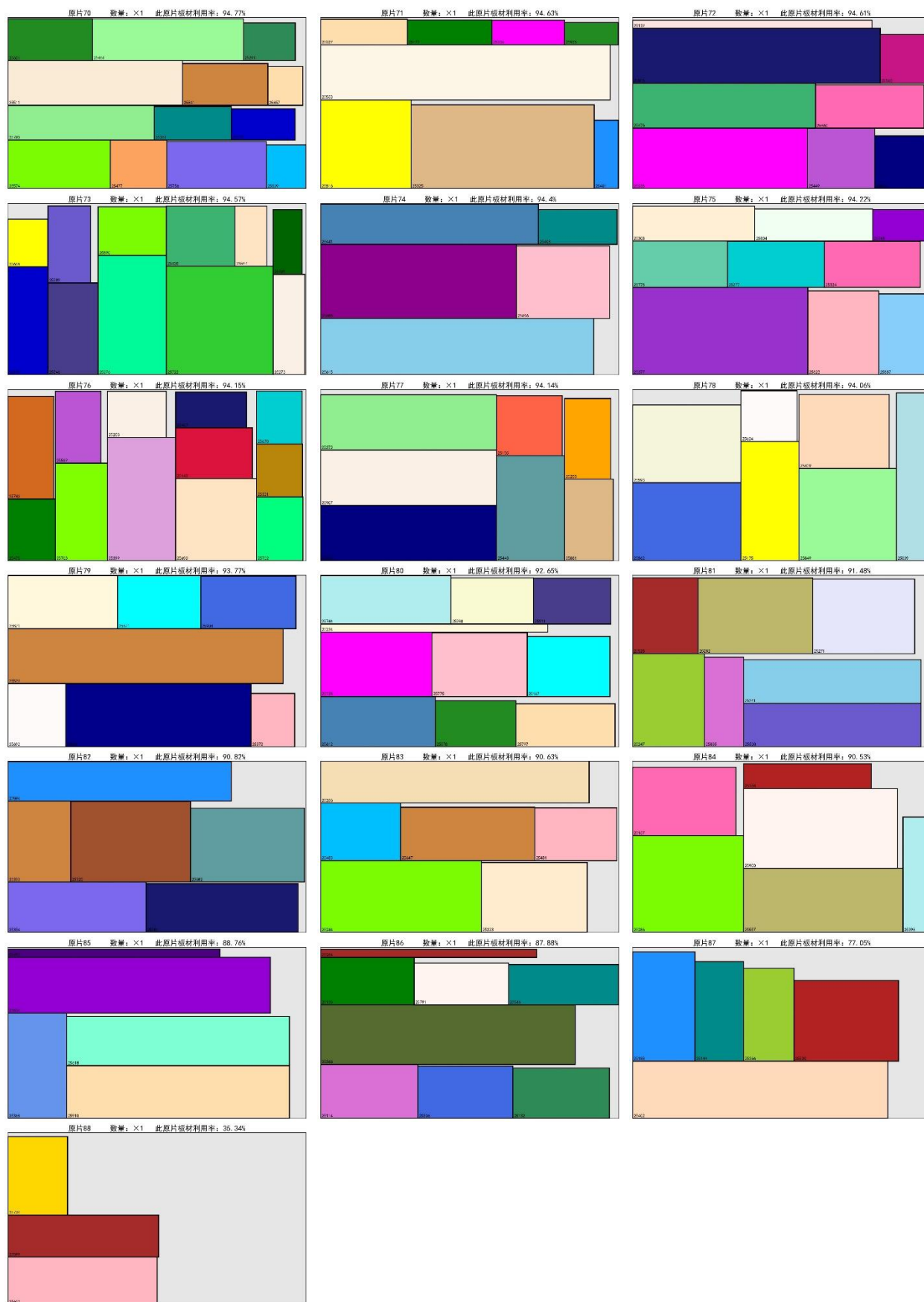
### 3) 数据集 A3







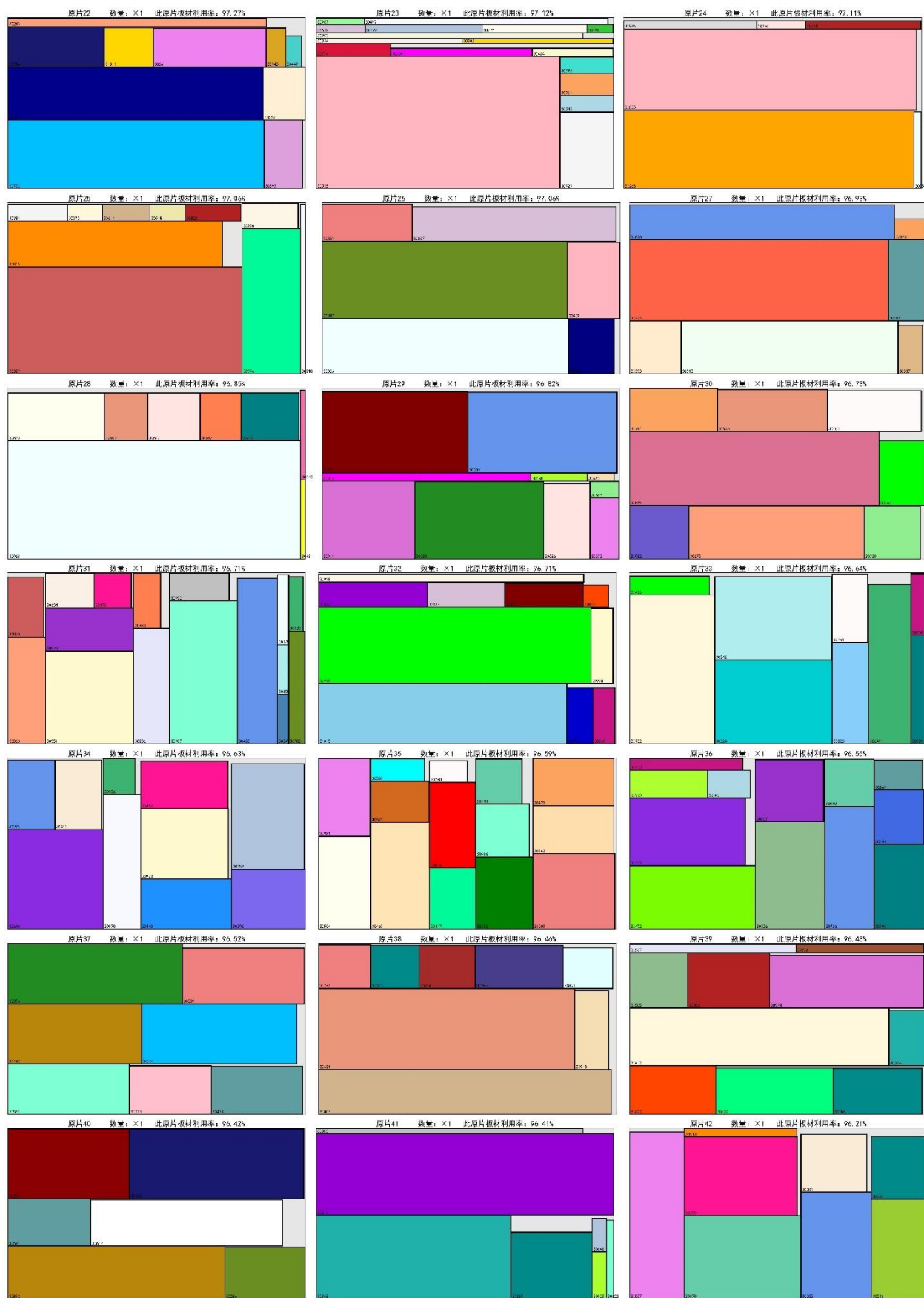


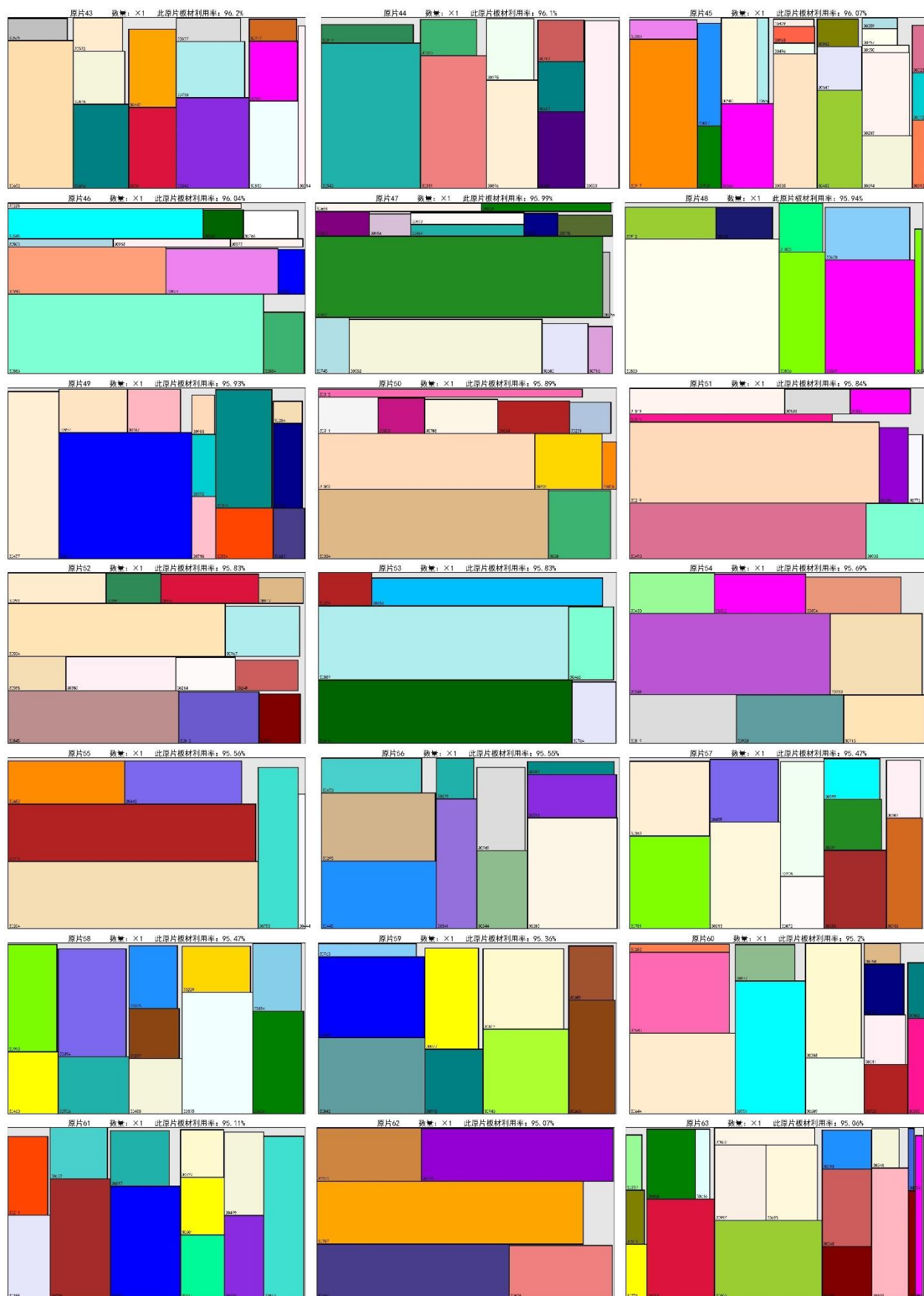


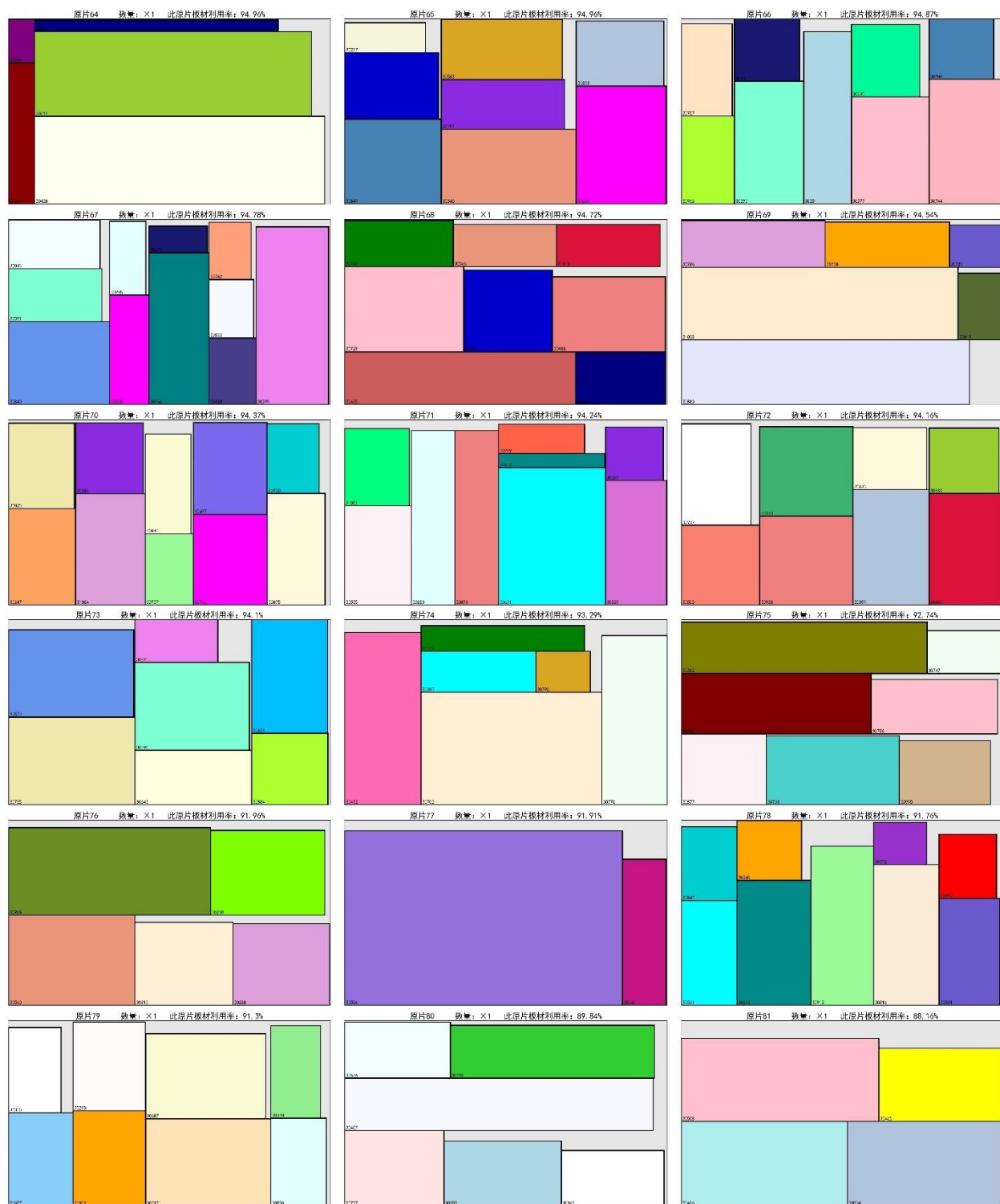


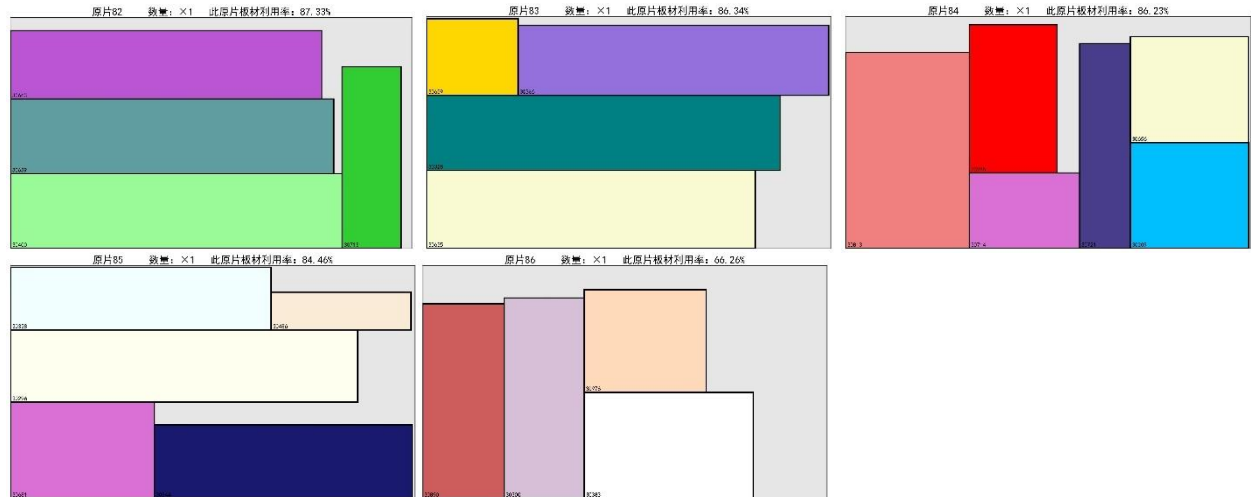
#### 4) 数据集 A4











## 9.2 代码及注释

### 1、产品项生成栈的函数代码

```
function stack=item_to_stack(item,L,W,rho)
%输入：item的所有信息:item,原片长度:L,原片宽度:W,价值修正系数:rho;
%输出：生成stack的所有信息:stack---一个三维数组
%初始参数设置
item_id=item(:,1);
item_length=item(:,3);
item_width=item(:,4);
item_square=item(:,2).*item(:,3);
item_value=rho.*item_square;
demand=item(:,5);
m=length(item(:,1));%item总数
lmin=[];
wmin=[];
for i=1:m
    lmin(i)=min(item_length(i:m));
    wmin(i)=min(item_width(i:m));
end
%以item i作为主item的stack,预先设定2m个
a=zeros(2*m+2,2);
a(3:2*m+2,1)=[1:2*m]';
for i=1:2*m
    stack(:, :, i)=a;
end
%通过循环进行item的放置
RL=L;%剩余高度
RW=W;%剩余宽度
demand0=demand;
for i=1:2*m
    if demand(i)==0
        return
    end
end
```

---

```

else
    %对每个stack进行item的放置i
    RL=L-item_length(i);
    RW=L-item_width(i);
    VL=0;%水平stack价值
    VW=0;%竖直stack价值
    ML=zeros(m,1);%存放每个item在水平stack放置的个数
    MW=zeros(m,1);%存放每个item在竖直stack放置的个数
    %水平放置stack
    demand1=demand;
    for j=1:2*m
        num_j=0;
        %stack均匀性要求
        if item_width(j)~=item_width(i)
            return
        end
        if demand1(j)==0 || item_length(j)>RL
            return
        end
        if item_length(j)>RL && RL<lmin(j)
            break
        end
        if RL<lmin(j) && RL>=lmin(j)
            return
        end
        %限制旋转前和旋转后的item只能使用一次
        if demand0-demand1(j)>=ML(j)
            return
        end
        if RL>lmin(j) && item_width(j)==item_width(i)
            if j<=m
                num_j=min(demand1(j), floor(RL/item_length(j)));
                ML(j)=ML(j)+1;
                RL=RL-num_j*item_length(j);
                VL=VL+num_j*item_value(j);
                demand1(j)=demand1(j)-1;
            else
                num_j=min(demand1(j-m), floor(RL/item_length(j-m)));
                ML(j-m)=ML(j-m)+1;
                RL=RL-num_j*item_length(j-m);
                VL=VL+num_j*item_value(j);
                demand1(j-m)=demand1(j-m)-1;
            end
        end
    end
end

```

---

```

end
%竖直放置stack
demand2=demand;
for j=1:2*m
    num_j=0;
    %stack均匀性要求
    if item_length(j)~=item_length(i)
        return
    end
    if demand2(j)==0 || item_width(j)>RW
        return
    end
    if item_width(j)>RW && RW<wmin(j)
        break
    end
    if RW<wmin(j) && RW>=wmin(j)
        return
    end
    %限制旋转前和旋转后的item只能使用一次
    if demand0-demand1(j)>=ML(j)
        return
    end
    if RW>wmin(j) && item_length(j)==item_length(i)
        if j<=m
            num_j=min(demand2(j), floor(RW/item_width(j)));
            MW(j)=MW(j)+1;
            RW=RW-num_j*item_width(j);
            VW=VW+num_j*item_value(j);
            demand2(j)=demand2(j)-1;
        else
            num_j=min(demand2(j-m), floor(RW/item_width(j-m)));
            MW(j-m)=MW(j-m)+1;
            RW=RW-num_j*item_width(j-m);
            VW=VW+num_j*item_value(j-m);
            demand2(j-m)=demand2(j-m)-1;
        end
    end
end
if VL>=VW
    a=zeros(m+2, 2);
    a(3:m+2, 1)=[1:m]';
    a(1, 1)=VL;%表示此stack的价值
    a(1, 2)=1;%表示此stack水平放置
    a(2, 1)=i;%表示此stack主item
    a(2, 2)=sum(ML.*item_length);%表示此stack的长度

```

---

```

        a(3:m+2, 2)=ML;%存放每个item放置的个数
        stack(:, :, i)=a;
        demand=demand1;
    else
        a=zeros(m+2, 2);
        a(3:m+2, 1)=[1:m]';
        a(1, 1)=VW;%表示此stack的价值
        a(1, 2)=2;%表示此stack竖直放置
        a(2, 1)=i;%表示此stack主item
        a(2, 2)=sum(MW.*item_width);%表示此stack的长度
        a(3:m+2, 2)=MW;%存放每个item放置的个数
        stack(:, :, i)=a;
        demand=demand2;
    end
end
end
end
end

```

## 2、栈生成条带的函数代码

```

function stripe=stack_to_stripe(item, stack, L, W)
%生成第k个stripe的函数
%输入: item的所有信息: item, stack的所有信息: stack, 原片长度: L, 原片宽度: W;
%输出: 生成stack的所有信息: stack--一个三维数组
%储存item的信息
item_id=item(:, 1);
item_length=item(:, 3);
item_width=item(:, 4);
item_value=rho*item_length.*item_width;
m=length(item(:, 1));%item总数
%计算stack的长度、宽度、和编号
stack_id=[];
stack_length=[];
stack_width=[];
for i=1:m
    ss=stack(:, :, i);
    if ss(2, 1)~=0
        stack_id=[stack_id; i];
        %判断已使用的stack的放置方式
        if ss(2, 2)==1
            stack_length=[stack_length; sum(item_length.*ss(3:m+2, 2))];
            stack_width=[stack_width; item_width(i)];
        else
            stack_length=[stack_length; item_length(i)];
            stack_width=[stack_width; sum(item_width.*ss(3:m+2, 2))];
        end
    end
end

```



---

```

        end
        stack_value=[stack_value;sum(item_value.*ss(3:m+2,2))];
    end
end
n=length(stack_id);
%类似于item组成stack的处理方式
slmin=[];
swmin=[];
for i=1:n
    slmin(i)=min(stack_length(i:n));
    swmin(i)=min(stack_width(i:n));
end
%每个stack的需求量
R=ones(1,n);
%允许一个stripe中的stack的高度不同,预先设定n个
a=zeros(n+2,2);
a(3:n+2,1)=[1:n]';
for i=1:n
    stripe(:, :, i)=a;
end
R0=R;
%循环放置
for k=1:n
    if R(i)==0
        return
    else
        VLL=0;%水平stripe价值
        VWW=0;%竖直stroke价值
        RLL=L-stack_length(k);
        RWW=W-stack_width(k);
        MLL=zeros(n,1);%存放每个stack在水平stripe放置的个数
        MWW=zeros(n,1);%存放每个stack在竖直stripe放置的个数
        %水平放置stripe
        dR1=R;
        for j=1:n
            if RLL>slmin(j) && stack_length(k)>=stack_length(j)
                MLL(j)=MLL(j)+1;
                RLL=RLL-stack_length(j);
                VLL=VLL+stack_value(j);
                dR1(j)=dR1(j)-1;
            end
        end
        %竖直放置stack
        dR2=R;

```

---

```

for j=1:n
    if RWW>swmin(j) && stack_width(k)>=stack_width(j)
        MWW(j)=MWW(j)+1;
        RWW=RWW-stack_width(j);
        VWW=VWW+stack_value(j);
        dR2(j)=dR2(j)-1;
    end
end
if VLL>=VWW
    a=zeros(n+2, 2);
    a(3:n+2, 1)=[1:n]';
    a(1, 1)=VLL;%表示此stripe的价值
    a(1, 2)=1;%表示此stripe水平放置
    a(2, 1)=k;%表示此stripe中最长stack
    a(2, 2)=sum(MLL.*stack_length);%表示此stripe的长度
    a(3:n+2, 2)=MLL;%存放每个stack放置的个数
    stripe(:, :, k)=a;
    R=dR1;
else
    a=zeros(n+2, 2);
    a(3:n+2, 1)=[1:n]';
    a(1, 1)=VWW;%表示此stripe的价值
    a(1, 2)=2;%表示此stripe竖直放置
    a(2, 1)=k;%表示此stripe最宽stack
    a(2, 2)=sum(MWW.*stack_width);%表示此stripe的宽度
    a(3:n+2, 2)=MWW;%存放每个stack放置的个数
    stripe(:, :, k)=a;
    R=dR2;
end
end
end
end

```

### 3、条带生成原片的函数代码

```

function plate=stripe_plate(item, stack, stripe, L, W)
%生成原片的函数
%输入： item的所有信息： item, stack的所有信息： stack, stripe的所有信息：stripe, 原片长度：
L, 原片宽度： W;
%输出： 生成plate的所有信息： plate--一个三维数组
%储存item的信息
item_id=item(:, 1);
item_length=item(:, 3);
item_width=item(:, 4);

```

---

```

item_value=rho*item_length.*item_width;
m=length(item(:,1));%item总数
%储存stack的信息
%计算stack的长度、宽度、和编号
stack_id=[];
stack_length=[];
stack_width=[];
for i=1:m
    ss=stack(:, :, i);
    if ss(2,1)~=0
        stack_id=[stack_id;i];
        %判断已使用的stack的放置方式
        if ss(2,2)==1
            stack_length=[stack_length;sum(item_length.*ss(3:m+2,2))];
            stack_width=[stack_width;item_width(i)];
        else
            stack_length=[stack_length;item_length(i)];
            stack_width=[stack_width;sum(item_width.*ss(3:m+2,2))];
        end
        stack_value=[stack_value;sum(item_value.*ss(3:m+2,2))]
    end
end
n=length(stack_id);
%计算stripe的长度、宽度、和编号
stripe_id=[];
stripe_length=[];
stripe_width=[];
for i=1:n
    ss=stripe(:, :, i);
    if ss(2,1)~=0
        stripe_id=[stripe_id;i];
        %判断已使用的stripe的放置方式
        if ss(2,2)==1
            stripe_length=[stripe_length;sum(stack_length.*ss(3:n+2,2))];
            stripe_width=[stripe_width;stack_width(i)];
        else
            stripe_length=[stripe_length;stack_length(i)];
            stripe_width=[stripe_width;sum(stripe_width.*ss(3:n+2,2))];
        end
        stripe_value=[stripe_value;sum(stack_value.*ss(3:n+2,2))]
    end
end
%stripe的个数
q=length(stripe_id);

```

---

```

%预先设定q个plate
a=zeros(q+2, 2);
a(3:q+2, 1)=[1:q]';
for i=1:n
    plate(:, :, i)=a;
end
R0=R;
%类似于item组成stack的处理方式
sslmin=[];
sswmin=[];
for i=1:q
    sslmin(i)=min(stripe_length(i:q));
    sswmin(i)=min(stripe_width(i:q));
end
%每个stripe的需求量
R=ones(1, q);
R0=R;
for l=1:q
    if R(i)==0
        return
    else
        RLLL=L-stripe_length(k);
        RWWW=W-stripe_width(k);
        MLLL=zeros(q, 1);%存放每个stripe在X向plate放置的个数
        MWWW=zeros(q, 1);%存放每个stack在Y向plate放置的个数
        %X向（第一阶段是水平切的）最优排样图生成
        %放置前l种条带得到的原片最大值
        Z1=zeros(q+1, W+1);
        Z1(:, 1)=zeros(q+1, 1);
        Z1(1, :)=zeros(1, q+1);
        for k=1:q
            for Y=1:W+1
                if Y>=stripe_width(k)
                    Z1(k+1, Y)=max(Z1(k, Y), Z1(k, Y-
floor(stripe_width(k)))+stripe_value(k));
                    MLLL(k)=MLLL(k)+1;
                    RWWW=RWWW-stripe_width(k);
                    dR1(k)=dR1(k)-1;
                    break
                end
            end
        end
        %Y向（第一阶段是竖直切的）最优排样图生成
        %放置前l种条带得到的原片最大值

```

---

```

Z2=zeros(q+1,W+1);
Z2(:,1)=zeros(q+1,1);
Z2(1,:)=zeros(1,q+1);
for k=1:q
    for Y=1:W+1
        if Y>=stripe_length(k)
            Z2(k+1,Y)=max(Z2(k,Y),Z2(k,Y-
floor(stripe_length(k))+stripe_value(k));
            MWWW(k)=MWWW(k)+1;
            RLLL=RLLL-stripe_length(k);
            dR2(k)=dR2(k)-1;
            break
        end
    end
end
if Z1(q+1,W+1)>=Z2(q+1,W+1)
    a=zeros(q+2,2);
    a(3:q+2,1)=[1:q]';
    a(1,1)=Z1(q+1,W+1);%表示此plate的价值
    a(1,2)=1;%表示此plateX向切割
    a(2,1)=k;%表示此plate第一条为stripe k
    a(2,2)=sum(MLLL.*stack_width);%表示此plate最后排满条带的高度
    a(3:n+2,2)=MLLL;%存放每个条带放置的个数
    plate(:, :, k)=a;
    R=dR1;
else
    a=zeros(n+2,2);
    a(3:q+2,1)=[1:q]';
    a(1,1)=Z2(q+1,W+1);%表示此stripe的价值
    a(1,2)=2;%表示此stripe竖直放置
    a(2,1)=k;%表示此plate第一条为stripe k
    a(2,2)=sum(MWWW.*stack_length);%表示此plate最后排满条带的长度
    a(3:n+2,2)=MWWW;%存放每个条带放置的个数
    plate(:, :, k)=a;
    R=dR2;
end
end
end
end

```

#### 4、HUSP生成当前排样图函数代码

```

function [stack, stripe, plate]=TUSP(item, L, W, rho)
%HUSP算法生成当前排样图

```

---

```

m=length(item(L, 1));
stack=item_to_stack(item, L, W, rho);
a=size(stack);
nn=a(3);
stripe=stack_to_stripe(item, stack, L, W);
b=size(stripe);
qq=b(3);
plate=stripe_plate(item, stack, stripe, L, W);
%记录当前生成的排样图中价值最高的
c=size(plate);
value=[];
for l=1:q(3)
    pp=plate(:, :, l);
    value_l=pp(1, 1);
    value=[value, value_l];
end
end
%最好排样图指标
id_op=find(value==max(value));
%记录最优排样图stripe
plate_op=plate(:, :, id_op);
stripe_output=plate(3:qq, 1:2);
stripe_used_id=find(plate_op(3:c(3), 2)~=0)-2;
stripe_used_id_demand=plate_op(stripe_used_id, 2);
%记录最优排样图的stack
ss=stripe(:, :, stripe(:, :stripe_used_id(1)));
stack_output_id=ss(3:nn, 1);
stack_output_id_demand=ss(3:nn, 2);
item_output_id=[1:m]';
for i=2:length(stripe_used_id)
    ss=stripe(:, :, stripe_used_id(i));
    s1=ss(3:nn, 2);
    stack_output_id_demand=stack_output_id_demand+d1;
end
stack_output=[stack_output_id, stack_output_id_demand];
%记录最优排样图的item
item_id=[1:m]';
item_demand=zeros(m, 1);
a=find(stack_output_id_demand~=0);
for i=1:length(a)
    aa=stack(:, :, a(i));
    item_demand=item_demand+aa(3:m, 2)
end
item_output=[item_id, item_demand];

```

---

```
end
```

## 5、主文件函数代码

```
%主文件——调用函数
```

```
clc
```

```
close all
```

```
clear all
```

```
%导入数据
```

```
[item_data, ~]=xlsread('data01.csv');
```

```
% [item_data, ~]=xlsread('data02.csv');
```

```
% [item_data, ~]=xlsread('data03.csv');
```

```
% [item_data, ~]=xlsread('data04.csv');
```

```
%存储每个item的编号、长度、宽度
```

```
item_id=item_data(:, 3);
```

```
item_length=item_data(:, 1);
```

```
item_width=item_data(:, 2);
```

```
freq=item_data(:, 4);
```

```
item=[item_id, item_length, item_width, freq];
```

```
%数据预处理
```

```
%记录长宽相同的item
```

```
y=[];
```

```
z=zeros(1, length(item_id));
```

```
for i=1:length(item_id)
```

```
    y=[y, i-1];
```

```
    for j=i+1:length(item_id)
```

```
        if item_length(j)==item_length(i) && item_width(j)==item_width(i)
```

```
            y=[y, j];
```

```
        end
```

```
    end
```

```
    y=[y, 1000];
```

```
end
```

```
%删除相同长宽的矩形项目
```

```
a=[];
```

```
i=1;
```

```
while 1
```

```
    if freq(i)~=1
```

```
        for j=1:freq(i)-1
```

```
            a=[a, i+j];
```

```
        end
```

```
    end
```

```
    if i+freq(i)>=length(freq)
```

```
        break
```

---

```

        end
        i=i+freq(i);
    end
    item(a,:)=[];

    %认为item的价值=item的面积*rho
    rho=item(:,2).*item(:,3);
    item_square=item(:,2).*item(:,3);
    item_value=rho.*item_square;

    %根据价值从小到大将item排列,
    [~,order]=sort(item_value,'ascend');
    item=[item(order,:),item_value(order)];

    %将item重新编号
    item=[[1:length(item(:,1))]',item];

    %使用SVC和HUSP算法求解3CS
    %参数初始化
    E_max=10000;%迭代次数;
    U_op=length(item);%最好排样方案中使用的原片个数
    t=item(:,6);%产品项初始单位价值
    for E=1:E_max
        c=item(:,5);%产品项的可用量
        U=0;
        J=0;
        while sum(c)>0
            %使用TUSP算法生成当前排样图
            [stack,stripe,plate]=TUSP(item,L,W,rho);
            %计算生成的排样图中item的种类和数目
            r=zeros(length(c),1);
            for i=1:length(c)
                a=item(:,i);
                aa=find(a(3:length(c),2)~=0);
                r(i)=length(aa);
            end
            %确定P的使用次数
            ff=[];
            for i=1:length(r)
                if r(i)~=0
                    b=c(i)/r(i);
                    ff=[ff,b];
                end
            end
        end
    end
end

```



---

```

f=min(ff);
%更新当前排样方案的排样方式数
J=J+1;
%更新原片使用数量
U=U+1;
%更新产品项可用量
for i=1:length(c)
    c(i)=c(i)-f*r(i);
end
%使用价值修正公式来修正产品项的价值
%价值修正公式参数设置
enta=0.75;
tao=1.02;
%计算排样图P中所含产品项的总面积和原片面积之比
s1=0;
item_length=item(:,3);
item_width=item(:,4);
item_square=item(:,2).*item(:,3);
for i=1:length(r)
    s1=s1+r(i)*item_square(i);
end
mu=s1/l/W;
for i=1:length(rho)
    lambda_1=1-enta*r(i)/(r(i)+d(i));
    lambda_2=1-lambda_1;
    rho(i)=lambda_1*rho(i)+(lambda_2*(item_square(i))^tao)/mu;
end
end
if U<U_op
    %记当前排样方案为最优方案
    U_op=U;
    J_OP=J;
end
if U_op==U && J_OP>J
    %记当前排样方案最优
    J_OP=J;
end
end
end

```

## 6、排样排版方案效果图绘制代码：

```

import matplotlib.pyplot as plt
import random
import pandas as pd

```

---

```
import numpy as np

from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'all'

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['font.family']='sans-serif'

### 颜色库

colorname = pd.Series(cnames)

len(cnames)

### 计算每个原片的利用率

import os
os.chdir(r'E: \dataA1')

df = pd.read_table('cut_program_A1.csv', sep=',')

df.head()

### Part 包含每个原片切割使用的板材面积，以及每个原片的板材利用率

df['area'] = df.x_length * df.y_length
Part = df.groupby('Pid').area.agg(['sum'])
Part['PartId'] = Part.index
Part['ratio'] = Part['sum']/(1220*2440)*100

## 计算总的板材利用率
sum(Part['sum'])/(1220*2440* max(df['Pid'])) *100

Part.ratio = round(Part.ratio,2)

# Part.to_csv(r'E: \dataA1\dataA1_板材利用率.csv',index=False)

plt.hist(Part.ratio)

### 绘制排样图

df.head(10)

PartNum = df.groupby('Pid').area.agg(['count'])
```

---

```

PartNum['Pid'] = PartNum.index

df.head(10)

for i in range(1,max(df.Pid)+1):
    print('没问题',i)
    fig = plt.figure(figsize=(17,10), dpi=200)
    ax = fig.add_subplot(111)

    ax = plt.subplot(facecolor='#e6e6e6') # 设置底色

    plt.xlim(0,2440)
    plt.ylim(0,1220)

    item_count = PartNum.loc[PartNum.Pid==i,'count'].reset_index(drop = True)

    randseries = random.sample(range(1,len(colurname)),item_count[0])
    randcolor = colurname[randseries]

    for k in range(item_count[0]):
        df_temp = df.loc[df.Pid==i].reset_index(drop = True)

        x_loc = df_temp['x_loc'][k]
        y_loc = df_temp['y_loc'][k]
        x_length = df_temp['x_length'][k]
        y_length = df_temp['y_length'][k]
        facecolor_rand = randcolor[k]
        itemid_temp = df_temp['itemid'][k]

        rect_temp = plt.Rectangle( (x_loc, y_loc), x_length, y_length, linewidth = 3,
                                   edgecolor = 'black', facecolor = facecolor_rand) # 坐
标, 长宽
        ax.add_patch(rect_temp)

        plt.annotate(itemid_temp, xy = (x_loc+10, y_loc+5), horizontalalignment = 'left',
verticalalignment = 'bottom', size=14)

Part_temp = Part[Part.PartId==i].reset_index(drop=True)
b = Part_temp.ratio[0]
plt.title('原片{} 数量: ×1 此原片板材利用率: {}%'.format(i,b), size=24)

```

---

```
plt.xticks([])
plt.yticks([])

tu_name = 'Part{}'.format(i)
plt.savefig('./tu/排样图_{}.jpg'.format(tu_name), format='jpg', dpi=200, bbox_inches='tight')
```

## 7、组批排版方案效果图绘制代码：

```
import matplotlib.pyplot as plt
import random
import pandas as pd
import numpy as np

from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'all'

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['font.family']='sans-serif'

#各颜色对应编码

colorname = pd.Series(cnames)

len(cnames)

# 计算每个原片的利用率

import os
os.chdir(r'E:\研究生\比赛\2022 届华为杯\数据代码 B')

df = pd.read_table('cut_programdataB1_English.csv', sep=',')

df.head()

# Part 包含每个原片切割使用的板材面积，以及每个原片的板材利用率

df['area'] = df.x_length * df.y_length

df['Pc+Pid'] = df['pc'].map(str) + '+' + df['Pid'].map(str)
df['PartSumArea'] = df.groupby('Pc+Pid').area.transform('sum')
df['ratio'] = df['PartSumArea']/(1220*2440)*100
```

---

```

PiciPart = df.loc[:,['pc', 'material', 'Pid', 'Pc+Pid', 'PartSumArea', 'ratio']]
PiciPart = PiciPart.drop_duplicates(keep='first', inplace=False)

PiciPart.shape

## 总原片数+总利用率

a = df['Pc+Pid'].unique()
len(a)

## 计算总的板材利用率
round(sum(PiciPart['PartSumArea'])/(1220*2440* len(a)) *100, 4)

PiciSum = PiciPart.groupby('pc').PartSumArea.agg(['sum'])
PiciSum['pc'] = PiciSum.index
PiciSum['PartNum'] = PiciPart.groupby('pc').pc.agg(['count'])
PiciSum['PcRatio'] = PiciSum['sum']/(2440*1220*PiciSum['PartNum'])*100

PiciPart.to_csv(r'E:\数据代码 B\板材利用率\dataB1_每原片_板材利用率.csv',index=False)
PiciSum.to_csv(r'E:\数据代码 B\板材利用率\dataB1_每批次_板材利用率.csv',index=False)

plt.hist(PiciPart.ratio)

plt.hist(PiciSum.PcRatio)

# 绘制排样图

df.head(10)

df['Pcount'] = df.groupby('Pc+Pid').area.transform('count')
#PartNum['Pc+Pid'] = PartNum.index

for l in range(1,max(df.pc)+1):
    for i in range(1,max(df.loc[df.pc==l,'Pid'])+1):
        print('没问题',i)
        fig = plt.figure(figsize=(17,10), dpi=80)
        ax = fig.add_subplot(111)

        ax = plt.subplot(facecolor='#e6e6e6') # 设置底色

        plt.xlim(0,2440)
        plt.ylim(0,1220)

        item_count = df.loc[(df.Pid==i)&(df.pc==l),'Pcount'].reset_index(drop = True)

```

---

```

randseries = random.sample(range(1,len(colurname)),item_count[0])
randcolor = colurname[randseries]

for k in range(item_count[0]):
    df_temp = df.loc[(df.Pid==i)&(df.pc==l)].reset_index(drop = True)

    x_loc = df_temp['x_loc'][k]
    y_loc = df_temp['y_loc'][k]
    x_length = df_temp['x_length'][k]
    y_length = df_temp['y_length'][k]
    facecolor_rand = randcolor[k]
    itemid_temp = df_temp['itemid'][k]

    rect_temp = plt.Rectangle( (x_loc, y_loc), x_length, y_length, linewidth = 3,
                                edgecolor = 'black', facecolor = facecolor_rand)    #
坐标, 长宽
    ax.add_patch(rect_temp)

    plt.annotate(itemid_temp, xy = (x_loc+10, y_loc+5), horizontalalignment = 'left',
verticalalignment = 'bottom', size=14)

Part_temp = PiciPart[(PiciPart.Pid==i)&(PiciPart.pc==l)].reset_index(drop=True)
b = round(Part_temp.ratio[0],2)
plt.title('批次{} 原片{}    数量: ×1   此原片板材利用率: {}'.format(l,i,b), size=24)

plt.xticks([])
plt.yticks([])

tu_name = 'Pici{}Part{}'.format(l,i)
plt.savefig('./tuB1/ 排 样 图 _{}.jpg'.format(tu_name),    format='jpg',    dpi=80,
bbox_inches='tight')

```